MODUL 06 · LISTS & SLIVERS

Scrollable Views & Optimasi Performa List Data

Menampilkan konten yang bisa digulir dengan SingleChildScrollView, ListView (biasa, builder, separated), ListTile, dan GridView; menambahkan interaksi swipe-to-delete, pull-to-refresh, infinite scroll, serta pencarian; menangani loading/empty state; dan menyusun tampilan lanjutan dengan CustomScrollView dan Slivers.

Durasi Pertemuan 6 (150 Menit) + Belajar Mandiri

Prasyarat Modul 05: Interaktivitas & Form

Capaian Pembelajaran

- Menggunakan SingleChildScrollView untuk halaman berisi konten terbatas dan mengatasi error overflow.
- Membedakan ListView biasa, ListView.builder, dan ListView.separated serta memilihnya sesuai jumlah data.
- Menyusun baris daftar yang informatif dengan ListTile.
- Menerapkan Dismissible (swipe to delete + urungkan) dan RefreshIndicator (pull-to-refresh).
- Membangun grid responsif dengan GridView.count dan GridView.builder.
- Menangani loading, error, dan empty state serta memuat data bertahap dengan ScrollController.
- Membuat pencarian dan filter daftar secara real-time dengan TextField.
- Mengimplementasikan CustomScrollView dengan SliverAppBar, SliverList, dan SliverGrid.

Daftar Isi Modul

1. SingleChildScrollView: Halaman yang Bisa Digulir
2. ListView, ListView.builder & ListView.separated
3. Ragam ListTile
4. Swipe to Delete dengan Dismissible
5. Pull-to-Refresh dengan RefreshIndicator
6. GridView.count & GridView.builder
7. Loading, Error & Empty State
8. Infinite Scroll dengan ScrollController
9. Pencarian & Filter Daftar
10. CustomScrollView, SliverAppBar, SliverList & SliverGrid
11. Studi Kasus: Daftar Mata Kuliah
12. Tugas Praktikum
13. Kuis Pemahaman & Kunci Jawaban

1. SingleChildScrollView: Halaman yang Bisa Digulir

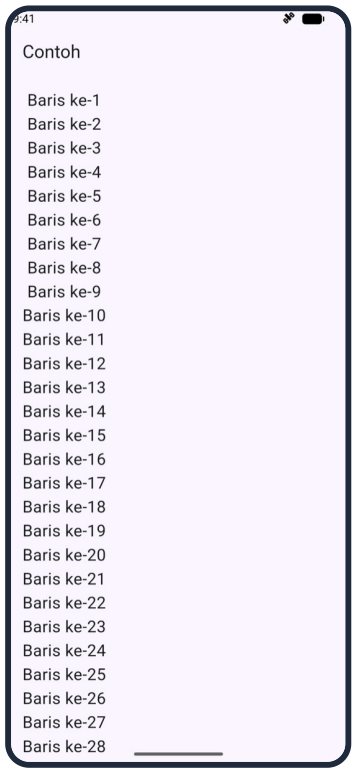
Layar HP itu pendek. Begitu isi halaman lebih tinggi dari layar, `Column` akan menampilkan garis kuning-hitam bertuliskan *"BOTTOM OVERFLOWED BY ... PIXELS"*. Solusi paling sederhana adalah `SingleChildScrollView`.

Konsep

- `SingleChildScrollView` membuat **satu** anak (biasanya `Column`) bisa digulir.
- Semua isi tetap dibuat sekaligus. Karena itu cocok untuk halaman yang isinya **terbatas**: profil, form, detail. Untuk ratusan data, gunakan `ListView.builder` (section berikutnya).
- `scrollDirection: Axis.horizontal` membuatnya bisa digulir ke samping, misalnya untuk baris chip.

Contoh kecil

```
// Column biasa + konten panjang = error "BOTTOM OVERFLOWED BY ... PIXELS"
// Solusi: bungkus Column dengan SingleChildScrollView
body: SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      for (var i = 1; i <= 30; i++) Text('Baris ke-$i', style: const TextStyle(fontSize: 20)),
    ],
  ),
),
```



30 baris teks yang bisa digulir tanpa overflow

Contoh lengkap: halaman profil mahasiswa

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const ProfilMahasiswaScreen(),
    );
  }
}

class ProfilMahasiswaScreen extends StatelessWidget {
  const ProfilMahasiswaScreen({super.key});

  static const _prestasi = [
    'Juara 1 Hackathon Fakultas',
    'Finalis Gemastik',
    'Asisten Praktikum Basis Data',
    'Ketua Panitia Seminar Nasional',
    'Penerima Beasiswa Prestasi',
  ];

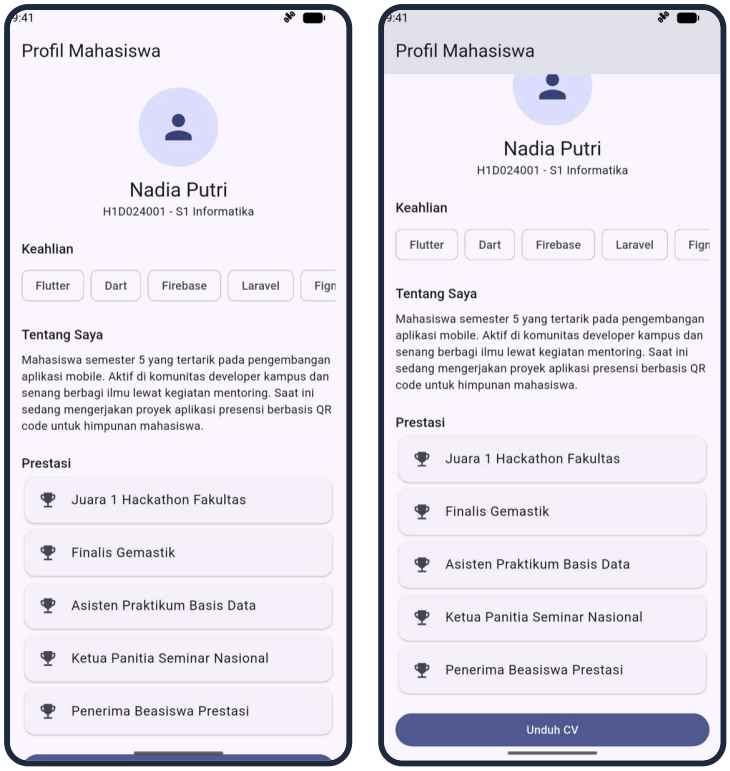
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Profil Mahasiswa')),
      // Seluruh halaman bisa digulir vertikal
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Center(
              child: CircleAvatar(radius: 48, child: Icon(Icons.person, size: 48)),
            ),
            const SizedBox(height: 12),
            Center(
              child: Text('Nadia Putri', style: Theme.of(context).textTheme.headlineSmall),
            ),
            const Center(child: Text('H1D024001 - S1 Informatika')),
            const SizedBox(height: 24),

            Text('Keahlian', style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            // Baris chip yang bisa digulir HORIZONTAL
```

```
            SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: Row(
                children: [
                  for (final k in ['Flutter', 'Dart', 'Firebase', 'Laravel', 'Figma', 'Git', 'SQL'])
                    Padding(
                      padding: const EdgeInsets.only(right: 8),
                      child: Chip(label: Text(k)),
                    ),
                ],
              ),
            ),
            const SizedBox(height: 24),

            Text('Tentang Saya', style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            const Text(
              'Mahasiswa semester 5 yang tertarik pada pengembangan aplikasi mobile. '
              'Aktif di komunitas developer kampus dan senang berbagi ilmu lewat '
              'kegiatan mentoring. Saat ini sedang mengerjakan proyek aplikasi '
              'presensi berbasis QR code untuk himpunan mahasiswa.',
            ),
            const SizedBox(height: 24),

            Text('Prestasi', style: Theme.of(context).textTheme.titleMedium),
            for (final p in _prestasi)
              Card(
                child: ListTile(leading: const Icon(Icons.emoji_events), title: Text(p)),
              ),
            const SizedBox(height: 16),
            SizedBox(
              width: double.infinity,
              child: FilledButton(onPressed: () {}, child: const Text('Unduh CV')),
            ),
          ],
        ),
      ),
    );
  }
}
```



Bagian atas halaman profil

Setelah digulir ke bawah

Penjelasan kode:

- `SingleChildScrollView` luar membuat seluruh halaman bisa digulir vertikal. `padding` bisa langsung diberikan padanya.
- `SingleChildScrollView` dalam memakai `scrollDirection: Axis.horizontal` untuk baris chip keahlian yang lebih lebar dari layar.
- Daftar prestasi dibuat dengan `for` di dalam `children`. Karena hanya 5 item, ini aman; tidak perlu ListView.

> ⚠️ **Kesalahan umum**
>
> - **Memakai `Expanded` atau `Spacer` di dalam Column yang dibungkus SingleChildScrollView.** Tinggi area gulir tidak terbatas sehingga Expanded tidak tahu harus seberapa besar, dan muncul error.
> - **Menaruh ListView di dalam SingleChildScrollView.** Dua area gulir bertumpuk membuat error "unbounded height". Pilih salah satu.
> - **Memakai SingleChildScrollView untuk ribuan data.** Semua item dibuat sekaligus dan aplikasi menjadi berat.

2. ListView, ListView.builder & ListView.separated

`ListView` adalah widget daftar yang bisa digulir. Ada tiga cara membuatnya, dan memilih yang tepat sangat berpengaruh pada kelancaran aplikasi.

ListView (children)

Membuat **seluruh item sekaligus** saat layar dibuka. Jika ada 1.000 data mahasiswa, Flutter membuat 1.000 widget meskipun hanya 8 yang terlihat. Cocok hanya untuk daftar pendek yang tetap, misalnya menu pengaturan.

ListView.builder (lazy)

Hanya membuat item yang **sedang terlihat di layar** ditambah sedikit cadangan. Item yang keluar layar dibuang. 100.000 data pun tetap ringan. Gunakan untuk data dari database atau API.

1. ListView biasa

```
body: ListView(
  children: const [
    ListTile(title: Text('Senin - Algoritma')),
    ListTile(title: Text('Selasa - Basis Data')),
    ListTile(title: Text('Rabu - Pemrograman Mobile')),
    ListTile(title: Text('Kamis - Jaringan Komputer')),
  ],
),
```



ListView biasa dengan empat ListTile

2. ListView.builder

Dua parameter wajib: `itemCount` (jumlah item) dan `itemBuilder` (fungsi yang membuat widget untuk posisi `index`).

```
final mahasiswa = List.generate(1000, (i) => 'Mahasiswa ke-${i + 1}');

return Scaffold(
  body: ListView.builder(
    itemCount: mahasiswa.length, // jumlah item
    itemBuilder: (context, index) {
      // Dipanggil HANYA untuk item yang akan tampil di layar
      return ListTile(
        leading: CircleAvatar(child: Text('${index + 1}')),
        title: Text(mahasiswa[index]),
      );
    },
  ),
);
```

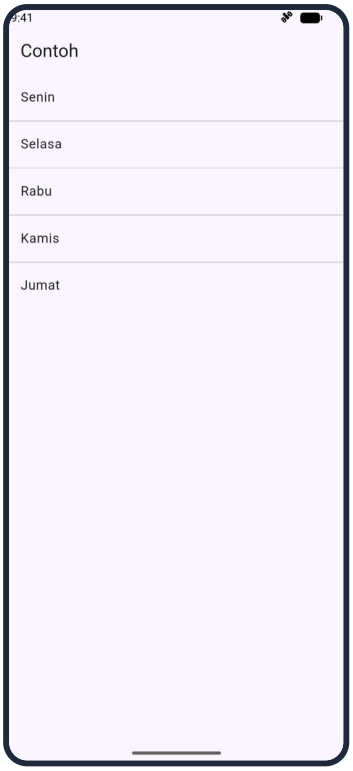


ListView.builder dengan 1000 item,
hanya yang terlihat yang dibuat

3. ListView.separated

Sama seperti builder, ditambah `separatorBuilder` untuk membuat pemisah di antara item.

```
body: ListView.separated(
  itemCount: hari.length,
  itemBuilder: (context, index) => ListTile(title: Text(hari[index])),
  // Pemisah dibuat DI ANTARA item (jumlahnya itemCount - 1)
  separatorBuilder: (context, index) => const Divider(height: 1),
),
```



Garis pemisah di antara item

Contoh lengkap: tiga jenis ListView dalam tab

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const PerbandinganListScreen(),
    );
  }
}

class PerbandinganListScreen extends StatelessWidget {
  const PerbandinganListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // DefaultTabController: tiga tab untuk tiga jenis ListView
    return DefaultTabController(
      length: 3,
      child: Scaffold(
```

```
        appBar: AppBar(
          title: const Text('Tiga Jenis ListView'),
          bottom: const TabBar(
            tabs: [
              Tab(text: 'ListView'),
              Tab(text: '.builder'),
              Tab(text: '.separated'),
            ],
          ),
        ),
        body: const TabBarView(
          children: [ListBiasa(), ListBuilder(), ListSeparated()],
        ),
      ),
    );
  }
}

// 1. ListView biasa: cocok untuk item SEDIKIT dan TETAP
class ListBiasa extends StatelessWidget {
  const ListBiasa({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(8),
      children: const [
        ListTile(leading: Icon(Icons.person), title: Text('Profil')),
        ListTile(leading: Icon(Icons.lock), title: Text('Ganti Password')),
        ListTile(leading: Icon(Icons.language), title: Text('Bahasa')),
        ListTile(leading: Icon(Icons.info), title: Text('Tentang Aplikasi')),
        ListTile(leading: Icon(Icons.logout), title: Text('Keluar')),
      ],
    );
  }
}

// 2. ListView.builder: cocok untuk data BANYAK / dinamis
class ListBuilder extends StatelessWidget {
  const ListBuilder({super.key});

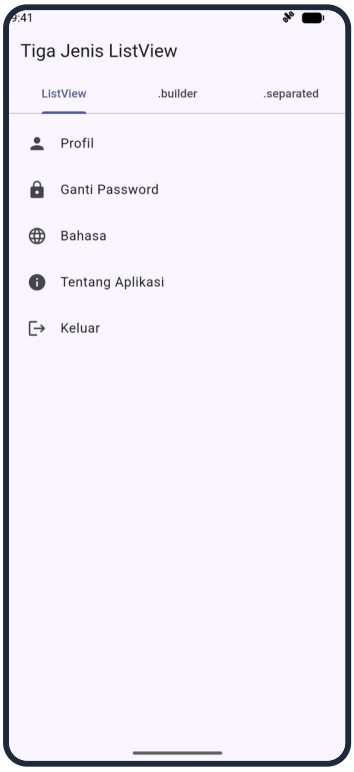
  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: 10000,
      itemBuilder: (context, index) {
        final nim = 'H1D0240${(index + 1).toString().padLeft(5, '0')}';
        return ListTile(
          leading: CircleAvatar(child: Text('${(index % 99) + 1}')),
          title: Text('Mahasiswa #${index + 1}'),
          subtitle: Text(nim),
        );
      },
    );
  }
}

// 3. ListView.separated: builder + pemisah otomatis
class ListSeparated extends StatelessWidget {
```

```dart
  const ListSeparated({super.key});

  static const _pengumuman = [
    'Batas pengisian KRS: 30 Agustus',
    'UTS dimulai minggu ke-8',
    'Libur nasional 17 Agustus',
    'Seminar karier di Aula Fakultas',
    'Pendaftaran asisten praktikum dibuka',
  ];

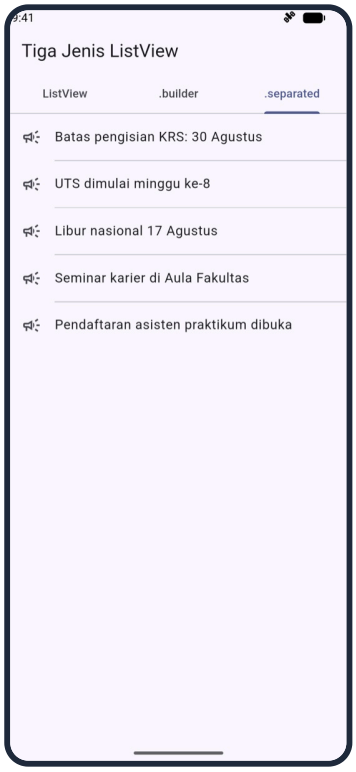
  @override
  Widget build(BuildContext context) {
    return ListView.separated(
      itemCount: _pengumuman.length,
      itemBuilder: (context, index) => ListTile(
        leading: const Icon(Icons.campaign_outlined),
        title: Text(_pengumuman[index]),
      ),
      separatorBuilder: (context, index) => const Divider(height: 1, indent: 56),
    );
  }
}
```



Tab ListView: menu pengaturan



Tab .builder: 10.000 mahasiswa tetap lancar



Tab .separated: pengumuman dengan pemisah

Penjelasan kode:

- `DefaultTabController` + `TabBar` + `TabBarView` membuat tiga tab. (Tab dibahas lebih lanjut di Modul 07.)
- Tab **ListView** berisi menu yang tetap dan pendek, jadi `children` biasa sudah tepat.

- Tab **.builder** berisi 10.000 item. Coba gulir cepat: tetap lancar karena hanya sekitar 10-15 ListTile yang benar-benar ada di memori.
- Tab **.separated** memakai `Divider(indent: 56)` agar garis dimulai sejajar teks, bukan ikon.

Situasi	Pilihan
Kurang dari ~20 item tetap (menu, pengaturan)	`ListView(children: ...)`
Data dari list/database/API, jumlah tidak pasti	`ListView.builder`
Seperti builder, perlu garis/jarak di antara item	`ListView.separated`

⚠️ **Kesalahan umum**

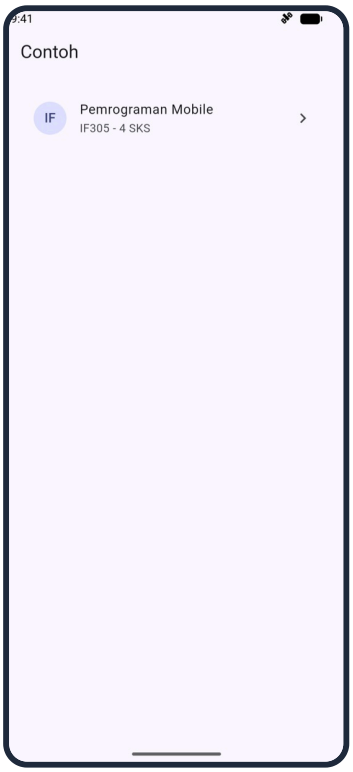
- **Error "Vertical viewport was given unbounded height".** Terjadi saat ListView diletakkan langsung di dalam `Column`. Bungkus ListView dengan `Expanded`.
- **Lupa `itemCount`.** Builder akan meminta item tanpa batas dan terjadi `RangeError` saat index melebihi panjang list.
- **Melakukan pekerjaan berat di `itemBuilder`** (misalnya mengurutkan seluruh list). itemBuilder dipanggil sangat sering saat menggulir; siapkan data di luar.

3. Ragam ListTile

`ListTile` adalah satu baris daftar siap pakai dengan slot `leading` (kiri), `title`, `subtitle`, dan `trailing` (kanan). Hampir semua aplikasi daftar (kontak, chat, pengaturan) bisa dibuat dengannya.

Contoh kecil

```
ListTile(
  leading: const CircleAvatar(child: Text('IF')),   // kiri
  title: const Text('Pemrograman Mobile'),           // judul
  subtitle: const Text('IF305 - 4 SKS'),             // keterangan
  trailing: const Icon(Icons.chevron_right),         // kanan
  onTap: () => debugPrint('Buka detail'),            // seluruh baris bisa ditekan
),
```



ListTile dengan avatar, judul, subjudul, dan ikon panah

Contoh lengkap: sepuluh ragam ListTile

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
```

```dart
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const KatalogListTileScreen(),
    );
  }
}

class KatalogListTileScreen extends StatefulWidget {
  const KatalogListTileScreen({super.key});

  @override
  State<KatalogListTileScreen> createState() => _KatalogListTileScreenState();
}

class _KatalogListTileScreenState extends State<KatalogListTileScreen> {
  int _terpilih = 1;
  bool _hadir = true;
  bool _wifi = false;

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

    return Scaffold(
      appBar: AppBar(title: const Text('Ragam ListTile')),
      body: ListView(
        children: [
          // 1. Paling dasar
          const ListTile(title: Text('1. Hanya judul')),

          // 2. Ikon + subjudul
          const ListTile(
            leading: Icon(Icons.event),
            title: Text('2. Ujian Tengah Semester'),
            subtitle: Text('Senin, 14 Oktober - Ruang C.2'),
          ),

          // 3. Avatar + trailing teks + bisa ditekan
          ListTile(
            leading: CircleAvatar(
              backgroundColor: warna.primaryContainer,
              child: const Text('NP'),
            ),
            title: const Text('3. Nadia Putri'),
            subtitle: const Text('Pesan: Besok kumpul jam 8 ya'),
            trailing: const Text('09:41'),
            onTap: () {},
          ),

          // 4. Tiga baris
          const ListTile(
            isThreeLine: true,
            leading: Icon(Icons.article_outlined),
            title: Text('4. Pengumuman Beasiswa'),
```

```
    subtitle: Text(
      'Pendaftaran beasiswa prestasi dibuka hingga akhir bulan. '
      'Syarat IPK minimal 3,25 dan aktif berorganisasi.',
    ),
  ),

  // 5. Badge di trailing
  ListTile(
    leading: const Icon(Icons.assignment),
    title: const Text('5. Tugas belum dikumpul'),
    trailing: Badge(
      label: const Text('3'),
      child: Icon(Icons.notifications, color: warna.primary),
    ),
  ),

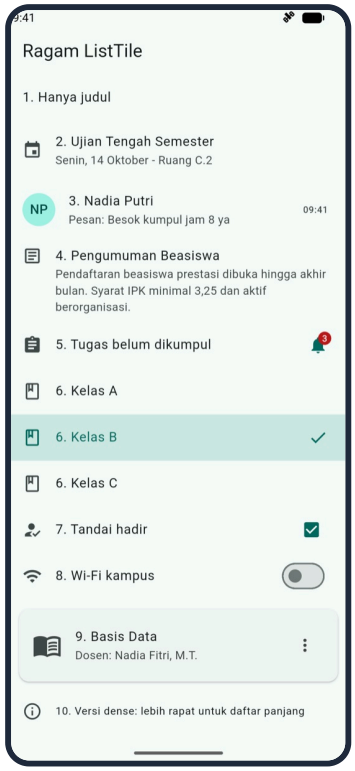
  // 6. Pilihan aktif (selected)
  for (var i = 0; i < 3; i++)
    ListTile(
      selected: _terpilih == i,
      selectedTileColor: warna.secondaryContainer,
      leading: const Icon(Icons.class_outlined),
      title: Text('6. Kelas ${String.fromCharCode(65 + i)}'),
      trailing: _terpilih == i ? const Icon(Icons.check) : null,
      onTap: () => setState(() => _terpilih = i),
    ),

  // 7. Versi siap pakai dengan checkbox / switch
  CheckboxListTile(
    secondary: const Icon(Icons.how_to_reg),
    title: const Text('7. Tandai hadir'),
    value: _hadir,
    onChanged: (v) => setState(() => _hadir = v ?? false),
  ),
  SwitchListTile(
    secondary: const Icon(Icons.wifi),
    title: const Text('8. Wi-Fi kampus'),
    value: _wifi,
    onChanged: (v) => setState(() => _wifi = v),
  ),

  // 9. Di dalam Card agar terlihat seperti kartu
  Card(
    margin: const EdgeInsets.all(12),
    child: ListTile(
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      leading: const Icon(Icons.menu_book, size: 36),
      title: const Text('9. Basis Data'),
      subtitle: const Text('Dosen: Nadia Fitri, M.T.'),
      trailing: IconButton(
        icon: const Icon(Icons.more_vert),
        onPressed: () {},
      ),
    ),
  ),

  // 10. Rapat (dense)
  const ListTile(
    dense: true,
```

```
            leading: Icon(Icons.info_outline),
            title: Text('10. Versi dense: lebih rapat untuk daftar panjang'),
          ),
        ],
      ),
    );
  }
}
```



Sepuluh ragam ListTile dalam satu layar

Penjelasan kode:

- **Nomor 4:** `isThreeLine: true` memberi ruang untuk subjudul dua baris.
- **Nomor 5:** `Badge` menampilkan angka kecil di atas ikon, cocok untuk jumlah notifikasi.
- **Nomor 6:** `selected` mengubah warna teks/ikon, dan `selectedTileColor` mengubah warna latar baris terpilih.
- **Nomor 7-8:** `CheckboxListTile` dan `SwitchListTile` sudah dibahas di Modul 05. Parameter `secondary` menaruh ikon di sisi berlawanan dari kontrol.
- **Nomor 9:** ListTile di dalam `Card` tampil seperti kartu. `trailing` bisa berisi tombol.
- **Nomor 10:** `dense: true` membuat baris lebih rapat.

⚠️ **Kesalahan umum**

- **Memberi `trailing` berupa `Row` tanpa `mainAxisSize: MainAxisSize.min`.** Row mengambil seluruh lebar dan terjadi error layout.
- **Memasang ListTile di dalam `Row`.** ListTile butuh lebar yang jelas; bungkus dengan `Expanded`.
- **Membungkus ListTile dengan InkWell/GestureDetector.** Tidak perlu, cukup isi `onTap`.

4. Swipe to Delete dengan Dismissible

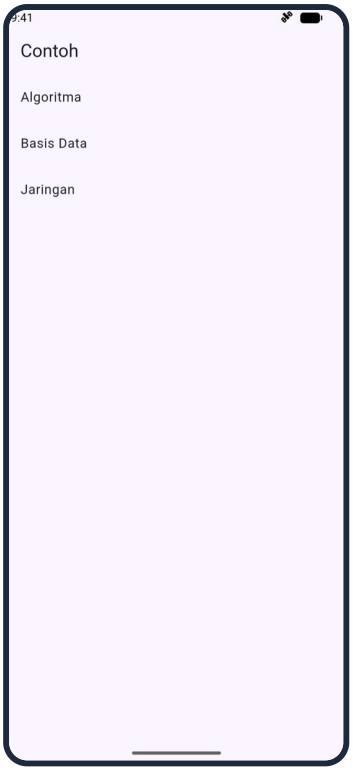
Menggeser baris ke kiri untuk menghapus (*swipe to delete*) adalah gestur yang sudah akrab bagi pengguna HP. Di Flutter, gestur ini dibuat dengan membungkus item daftar memakai `Dismissible`.

Konsep

- `key` : **wajib** dan harus unik untuk setiap item, biasanya dari id data: `ValueKey(item.id)`.
- `background` : tampilan di balik item saat digeser ke kanan (atau ke dua arah jika `secondaryBackground` tidak diisi). `secondaryBackground` : tampilan saat digeser ke kiri.
- `direction` : membatasi arah geser.
- `confirmDismiss` : fungsi `async` yang dipanggil *sebelum* item hilang. Kembalikan `false` untuk membatalkan.
- `onDismissed` : dipanggil setelah item hilang dari layar. Di sini data **wajib** dihapus dari list.

Contoh kecil

```
itemBuilder: (context, index) {
  final item = _items[index];
  return Dismissible(
    key: ValueKey(item), // WAJIB unik dan stabil untuk tiap item
    direction: DismissDirection.endToStart, // hanya geser ke kiri
    background: Container(
      color: Colors.red,
      alignment: Alignment.centerRight,
      padding: const EdgeInsets.only(right: 20),
      child: const Icon(Icons.delete, color: Colors.white),
    ),
    onDismissed: (direction) {
      // WAJIB hapus data dari list, kalau tidak akan error
      setState(() => _items.removeAt(index));
    },
    child: ListTile(title: Text(item)),
  );
},
```



Daftar sederhana; setiap baris bisa digeser ke kiri untuk dihapus

Contoh lengkap: kotak masuk dengan hapus, urungkan, dan arsip

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KotakMasukScreen(),
    );
  }
}

class Pesan {
  final int id;
  final String pengirim;
  final String isi;
  bool diarsipkan;

  Pesan(this.id, this.pengirim, this.isi, {this.diarsipkan = false});
}
```

```dart
class KotakMasukScreen extends StatefulWidget {
  const KotakMasukScreen({super.key});

  @override
  State<KotakMasukScreen> createState() => _KotakMasukScreenState();
}

class _KotakMasukScreenState extends State<KotakMasukScreen> {
  final List<Pesan> _pesan = [
    Pesan(1, 'Akademik', 'Jadwal UTS sudah terbit'),
    Pesan(2, 'Dosen Wali', 'Silakan konsultasi KRS minggu ini'),
    Pesan(3, 'Himpunan', 'Rapat anggota hari Jumat'),
    Pesan(4, 'Perpustakaan', 'Buku yang dipinjam jatuh tempo besok'),
    Pesan(5, 'BEM', 'Open recruitment panitia dies natalis'),
    Pesan(6, 'Lab Komputer', 'Praktikum minggu ini diganti online'),
  ];

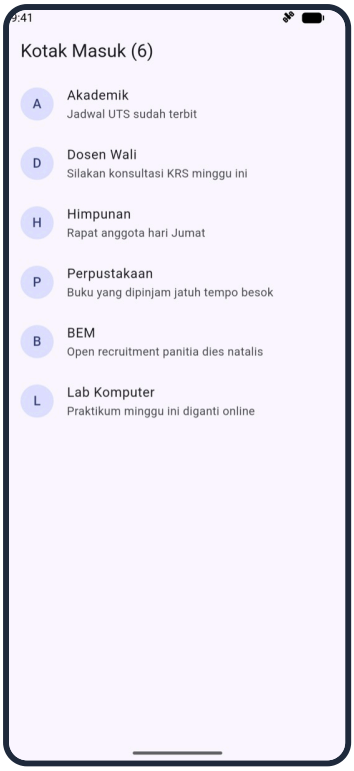
  void _hapus(int index) {
    final terhapus = _pesan[index];
    setState(() => _pesan.removeAt(index));

    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          content: Text('Pesan dari ${terhapus.pengirim} dihapus'),
          action: SnackBarAction(
            label: 'URUNGKAN',
            // Kembalikan ke posisi semula
            onPressed: () => setState(() => _pesan.insert(index, terhapus)),
          ),
        ),
      );
  }

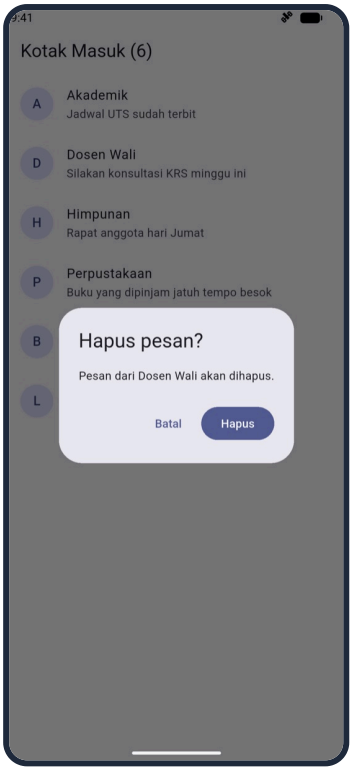
  Future<bool> _konfirmasiHapus(Pesan p) async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Hapus pesan?'),
        content: Text('Pesan dari ${p.pengirim} akan dihapus.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );
    return yakin ?? false;
  }

  @override
  Widget build(BuildContext context) {
```

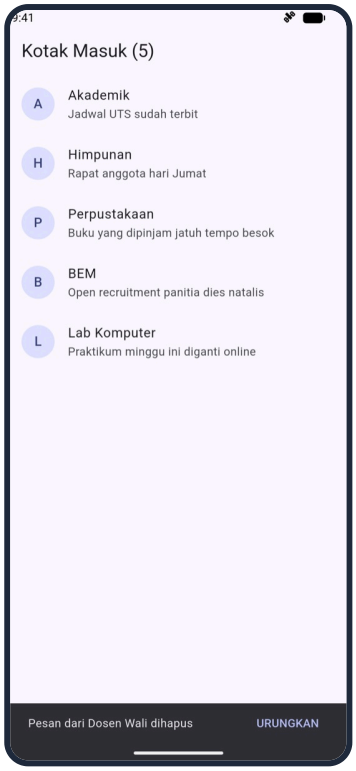
```
    return Scaffold(
      appBar: AppBar(title: Text('Kotak Masuk (${_pesan.length})')),
      body: _pesan.isEmpty
          ? const Center(child: Text('Kotak masuk kosong'))
          : ListView.builder(
              itemCount: _pesan.length,
              itemBuilder: (context, index) {
                final p = _pesan[index];
                return Dismissible(
                  // Pakai id unik dari data, BUKAN index
                  key: ValueKey(p.id),
                  // Geser ke kanan -> arsip (latar hijau)
                  background: Container(
                    color: Colors.green,
                    alignment: Alignment.centerLeft,
                    padding: const EdgeInsets.only(left: 24),
                    child: const Icon(Icons.archive, color: Colors.white),
                  ),
                  // Geser ke kiri -> hapus (latar merah)
                  secondaryBackground: Container(
                    color: Colors.red,
                    alignment: Alignment.centerRight,
                    padding: const EdgeInsets.only(right: 24),
                    child: const Icon(Icons.delete, color: Colors.white),
                  ),
                  // Dipanggil SEBELUM item hilang. Return false = batal.
                  confirmDismiss: (direction) async {
                    if (direction == DismissDirection.startToEnd) {
                      // Arsip: ubah status saja, item tidak dihapus dari list
                      setState(() => p.diarsipkan = !p.diarsipkan);
                      return false;
                    }
                    return _konfirmasiHapus(p);
                  },
                  onDismissed: (_) => _hapus(index),
                  child: ListTile(
                    leading: CircleAvatar(child: Text(p.pengirim[0])),
                    title: Text(p.pengirim),
                    subtitle: Text(p.isi),
                    trailing: p.diarsipkan
                        ? const Icon(Icons.archive, color: Colors.green)
                        : null,
                  ),
                );
              },
            ),
    );
  }
}
```



Daftar pesan awal



Pesan digeser ke kiri: dialog konfirmasi muncul



Setelah dihapus: SnackBar dengan tombol URUNGKAN

Penjelasan kode:

- `key: ValueKey(p.id)` memakai id unik dari data. Jika memakai `index`, setelah satu item dihapus semua index bergeser dan Flutter bisa salah mengenali item.
- Geser ke **kanan** (`startToEnd`) hanya mengubah status arsip lalu `return false`, sehingga item kembali ke tempatnya.
- Geser ke **kiri** memanggil `_konfirmasiHapus` yang menampilkan dialog. Hasil dialog (true/false) langsung menjadi hasil `confirmDismiss`.
- `_hapus` menyimpan item dan posisinya sebelum menghapus, lalu tombol URUNGKAN memasukkannya kembali dengan `insert`.

> ⚠️ **Kesalahan umum**
>
> - **Error "A dismissed Dismissible widget is still part of the tree".** Terjadi jika `onDismissed` tidak menghapus data dari list. Selalu `removeAt` / `remove` lalu `setState`.
> - **Key memakai index atau nama yang bisa kembar.** Gunakan id unik.
> - **Menghapus permanen ke database langsung di `onDismissed` padahal ada tombol Urungkan.** Pada aplikasi nyata, tunda penghapusan permanen sampai SnackBar hilang, atau kembalikan data ke database saat diurungkan.

5. Pull-to-Refresh dengan RefreshIndicator

Menarik daftar ke bawah untuk memuat ulang (*pull to refresh*) dibuat dengan membungkus widget gulir memakai `RefreshIndicator`.

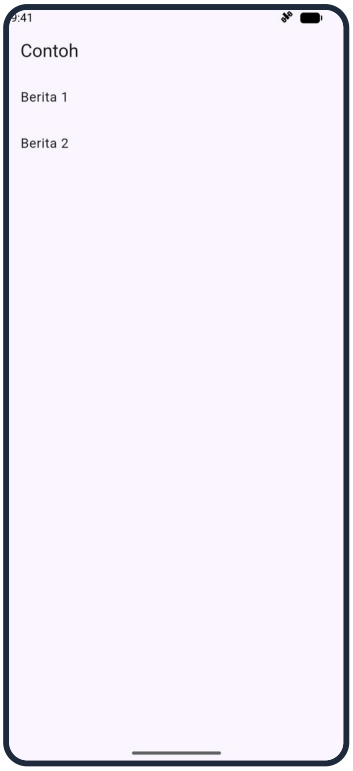
Konsep

- `onRefresh` harus berupa fungsi yang mengembalikan `Future`. Lingkaran loading terus berputar sampai Future selesai.
- Anak RefreshIndicator harus widget yang bisa digulir (ListView, GridView, CustomScrollView).
- Jika item sedikit dan tidak memenuhi layar, daftar tidak bisa ditarik. Tambahkan `physics: const AlwaysScrollableScrollPhysics()`.

Contoh kecil

```
// onRefresh WAJIB mengembalikan Future: indikator berputar sampai Future selesai
Future<void> _muatUlang() async {
  await Future.delayed(const Duration(seconds: 1)); // pura-pura ambil dari server
  setState(() => _berita = ['Berita terbaru', ..._berita]);
}

@override
Widget build(BuildContext context) {
  return RefreshIndicator(
    onRefresh: _muatUlang,
    child: ListView.builder(
      // agar tetap bisa ditarik walau item sedikit
      physics: const AlwaysScrollableScrollPhysics(),
      itemCount: _berita.length,
      itemBuilder: (context, index) => ListTile(title: Text(_berita[index])),
    ),
  );
}
```



Daftar berita di dalam RefreshIndicator

Contoh lengkap: katalog mata kuliah dengan pull-to-refresh

lib/main.dart

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DaftarMataKuliahScreen(),
    );
  }
}


class DaftarMataKuliahScreen extends StatefulWidget {
  const DaftarMataKuliahScreen({super.key});

  @override
  State<DaftarMataKuliahScreen> createState() => _DaftarMataKuliahScreenState();
}

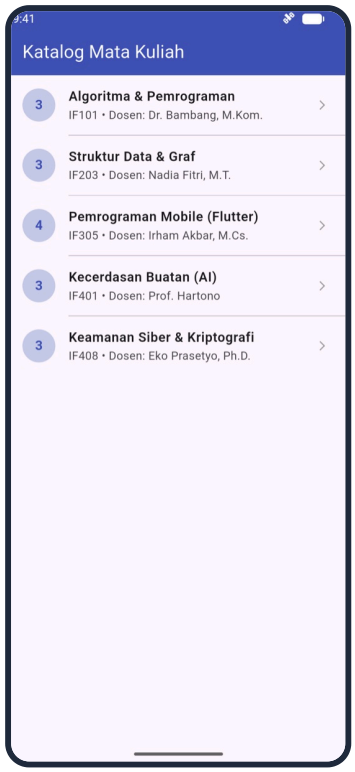
class _DaftarMataKuliahScreenState extends State<DaftarMataKuliahScreen> {
```

```dart
  // Simulasi data dari database kampus
  final List<Map<String, dynamic>> _daftarMatkul = [
    {"kode": "IF101", "nama": "Algoritma & Pemrograman", "sks": 3, "dosen": "Dr. Bambang, M.Kom."},
    {"kode": "IF203", "nama": "Struktur Data & Graf", "sks": 3, "dosen": "Nadia Fitri, M.T."},
    {"kode": "IF305", "nama": "Pemrograman Mobile (Flutter)", "sks": 4, "dosen": "Irham Akbar,
M.Cs."},
    {"kode": "IF401", "nama": "Kecerdasan Buatan (AI)", "sks": 3, "dosen": "Prof. Hartono"},
    {"kode": "IF408", "nama": "Keamanan Siber & Kriptografi", "sks": 3, "dosen": "Eko Prasetyo,
Ph.D."},
  ];

  // Fungsi simulasi refresh data dari server
  Future<void> _refreshData() async {
    await Future.delayed(const Duration(seconds: 2));
    setState(() {
      _daftarMatkul.shuffle(); // Acak urutan sebagai tanda data baru
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Katalog Mata Kuliah"),
        backgroundColor: Colors.indigo,
        foregroundColor: Colors.white,
      ),
      // RefreshIndicator menambahkan gestur tarik ke bawah untuk refresh
      body: RefreshIndicator(
        onRefresh: _refreshData,
        // AlwaysScrollableScrollPhysics: tarik-ke-bawah tetap bisa walau item sedikit
        child: ListView.separated(
          physics: const AlwaysScrollableScrollPhysics(),
          itemCount: _daftarMatkul.length,
          separatorBuilder: (context, index) => const Divider(height: 1, indent: 72),
          itemBuilder: (context, index) {
            final matkul = _daftarMatkul[index];
            return ListTile(
              leading: CircleAvatar(
                backgroundColor: Colors.indigo.shade100,
                child: Text(
                  "${matkul['sks']}",
                  style: const TextStyle(color: Colors.indigo, fontWeight: FontWeight.bold),
                ),
              ),
              title: Text(
                matkul['nama'],
                style: const TextStyle(fontWeight: FontWeight.w600),
              ),
              subtitle: Text("${matkul['kode']} • Dosen: ${matkul['dosen']}"),
              trailing: const Icon(Icons.arrow_forward_ios, size: 14, color: Colors.grey),
              onTap: () {
                // Aksi saat item baris ditekan
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text("Membuka ${matkul['nama']}")),
                );
              },
            );
          },
```

```
        ),
      ),
    );
  }
}
```



Katalog mata kuliah dengan ListView.separated. Tarik ke bawah untuk memuat ulang

Penjelasan kode:

- `_refreshData` menunggu 2 detik (pura-pura mengambil data dari server), lalu mengacak urutan sebagai tanda data berubah.
- `ListView.separated` dipakai agar setiap mata kuliah dipisah garis tipis.
- `CircleAvatar` di `leading` menampilkan jumlah SKS.

> 💡 **Tips**
>
> Pada Modul 09 (REST API), isi `onRefresh` akan diganti dengan pemanggilan `http.get` sungguhan. Polanya sama persis: `await` ambil data, lalu `setState`.

⚠️ **Kesalahan umum**

- **onRefresh tidak `async` / tidak mengembalikan Future.** Indikator langsung hilang atau terjadi error tipe.
- **Lupa memeriksa `mounted`** setelah `await` pada data sungguhan; jika pengguna sudah keluar halaman, `setState` akan error.
- **RefreshIndicator dibungkuskan ke Column.** Ia harus langsung membungkus widget yang bisa digulir.

6. GridView.count & GridView.builder

Jika ListView menyusun item ke bawah, `GridView` menyusunnya dalam baris dan kolom. Cocok untuk menu ikon, galeri foto, atau katalog produk.

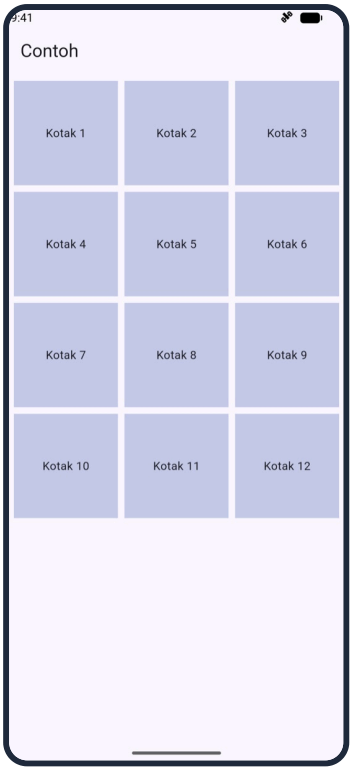
Konsep: dua cara utama

Konstruktor	Cara kerja	Cocok untuk
`GridView.count`	Jumlah kolom tetap (`crossAxisCount`), semua anak dibuat sekaligus	Menu dengan item sedikit
`GridView.builder`	Lazy seperti ListView.builder, tata letak diatur `gridDelegate`	Data banyak / dinamis

`childAspectRatio` adalah perbandingan lebar : tinggi tiap kotak. Nilai 1 = persegi, kurang dari 1 = lebih tinggi, lebih dari 1 = lebih lebar.

Contoh kecil: GridView.count

```
body: GridView.count(
  crossAxisCount: 3,       // 3 kolom
  mainAxisSpacing: 8,      // jarak vertikal antar kotak
  crossAxisSpacing: 8,     // jarak horizontal antar kotak
  padding: const EdgeInsets.all(8),
  children: [
    for (var i = 1; i <= 12; i++)
      Container(
        color: Colors.indigo.shade100,
        alignment: Alignment.center,
        child: Text('Kotak $i'),
      ),
  ],
),
```



GridView tiga kolom berisi 12 kotak

Contoh lengkap 1: menu akademik dengan GridView.count

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const MenuAkademikScreen(),
    );
  }
}

class MenuItem {
  final String label;
  final IconData ikon;
  final Color warna;

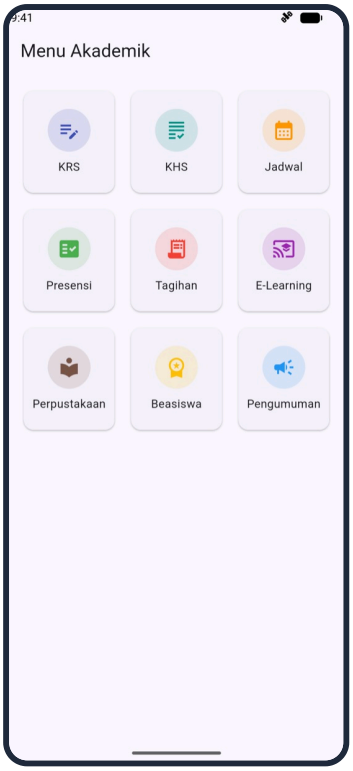
  const MenuItem(this.label, this.ikon, this.warna);
}

class MenuAkademikScreen extends StatelessWidget {
```

```dart
  const MenuAkademikScreen({super.key});

  static const _menu = [
    MenuItem('KRS', Icons.edit_note, Colors.indigo),
    MenuItem('KHS', Icons.grading, Colors.teal),
    MenuItem('Jadwal', Icons.calendar_month, Colors.orange),
    MenuItem('Presensi', Icons.fact_check, Colors.green),
    MenuItem('Tagihan', Icons.receipt_long, Colors.red),
    MenuItem('E-Learning', Icons.cast_for_education, Colors.purple),
    MenuItem('Perpustakaan', Icons.local_library, Colors.brown),
    MenuItem('Beasiswa', Icons.workspace_premium, Colors.amber),
    MenuItem('Pengumuman', Icons.campaign, Colors.blue),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Menu Akademik')),
      body: GridView.count(
        crossAxisCount: 3,
        padding: const EdgeInsets.all(16),
        mainAxisSpacing: 12,
        crossAxisSpacing: 12,
        childAspectRatio: 0.9, // lebar : tinggi = 0.9 (sedikit lebih tinggi)
        children: [
          for (final m in _menu)
            Card(
              clipBehavior: Clip.antiAlias,
              child: InkWell(
                onTap: () => ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text('Membuka menu ${m.label}')),
                ),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    CircleAvatar(
                      radius: 26,
                      backgroundColor: m.warna.withValues(alpha: 0.15),
                      child: Icon(m.ikon, color: m.warna, size: 28),
                    ),
                    const SizedBox(height: 8),
                    Text(m.label, textAlign: TextAlign.center),
                  ],
                ),
              ),
            ),
        ],
      ),
    );
  }
}
```



Menu akademik 3 kolom dengan ikon berwarna

- Setiap menu berupa `Card` + `InkWell` sehingga ada efek riak saat ditekan.
- `m.warna.withValues(alpha: 0.15)` membuat latar ikon transparan 15%.

Contoh lengkap 2: katalog buku dengan GridView.builder

`lib/main.dart`

```
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepOrange),
      home: const KatalogBukuScreen(),
    );
  }
}

class KatalogBukuScreen extends StatefulWidget {
  const KatalogBukuScreen({super.key});

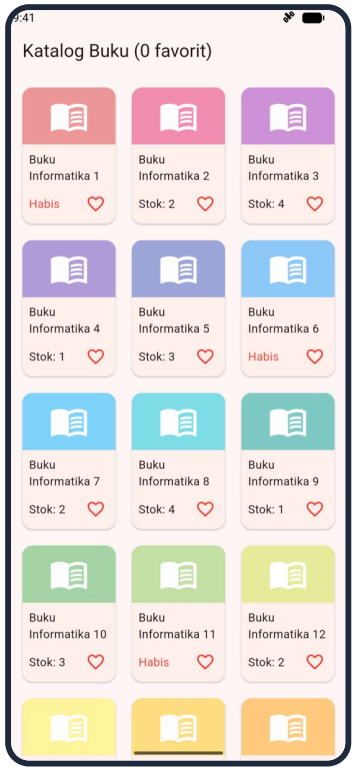
  @override
```

```
  State<KatalogBukuScreen> createState() => _KatalogBukuScreenState();
}

class _KatalogBukuScreenState extends State<KatalogBukuScreen> {
  // 60 buku contoh, dibuat otomatis
  final _buku = List.generate(
    60,
    (i) => (judul: 'Buku Informatika ${i + 1}', stok: (i * 7) % 5),
  );
  final Set<int> _favorit = {};

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Katalog Buku (${_favorit.length} favorit)')),
      body: GridView.builder(
        padding: const EdgeInsets.all(12),
        itemCount: _buku.length,
        // Lebar maksimum tiap kotak 180 px: jumlah kolom menyesuaikan lebar layar
        gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
          maxCrossAxisExtent: 180,
          mainAxisSpacing: 12,
          crossAxisSpacing: 12,
          childAspectRatio: 0.7,
        ),
        itemBuilder: (context, index) {
          final b = _buku[index];
          final favorit = _favorit.contains(index);
          return Card(
            clipBehavior: Clip.antiAlias,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Expanded(
                  child: Container(
                    color: Colors.primaries[index % Colors.primaries.length].shade200,
                    child: const Icon(Icons.menu_book, size: 48, color: Colors.white),
                  ),
                ),
                Padding(
                  padding: const EdgeInsets.fromLTRB(8, 8, 8, 0),
                  child: Text(b.judul, maxLines: 2, overflow: TextOverflow.ellipsis),
                ),
                Row(
                  children: [
                    const SizedBox(width: 8),
                    Expanded(
                      child: Text(
                        b.stok == 0 ? 'Habis' : 'Stok: ${b.stok}',
                        style: TextStyle(color: b.stok == 0 ? Colors.red : null),
                      ),
                    ),
                    IconButton(
                      icon: Icon(favorit ? Icons.favorite : Icons.favorite_border),
                      color: Colors.red,
                      onPressed: () => setState(() {
                        favorit ? _favorit.remove(index) : _favorit.add(index);
                      }),
                    ),
```

```
                ],
              ),
            ],
          ),
        );
      },
    ),
  );
 }
}
```



Katalog 60 buku dengan GridView.builder

Penjelasan kode:

- `SliverGridDelegateWithMaxCrossAxisExtent(maxCrossAxisExtent: 180)` artinya "setiap kotak maksimal 180 piksel". Di HP muncul 2 kolom, di tablet bisa 4-5 kolom secara otomatis. Inilah cara mudah membuat grid responsif.
- Alternatifnya, `SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 2)` memberi jumlah kolom tetap.
- Data buku dibuat dengan `List.generate` berisi *record* `(judul: ..., stok: ...)`.
- Favorit disimpan di `Set<int>` berisi index buku.

⚠️ **Kesalahan umum**

- **Garis kuning overflow di dalam kotak grid.** Tinggi kotak ditentukan oleh `childAspectRatio`, bukan oleh isinya. Kecilkan isi atau ubah rasio (misalnya 0.7).
- **GridView di dalam Column tanpa `Expanded`** : error unbounded height, sama seperti ListView.
- **Memakai GridView.count untuk ratusan item**: semua kotak dibuat sekaligus. Pakai builder.

7. Loading, Error & Empty State

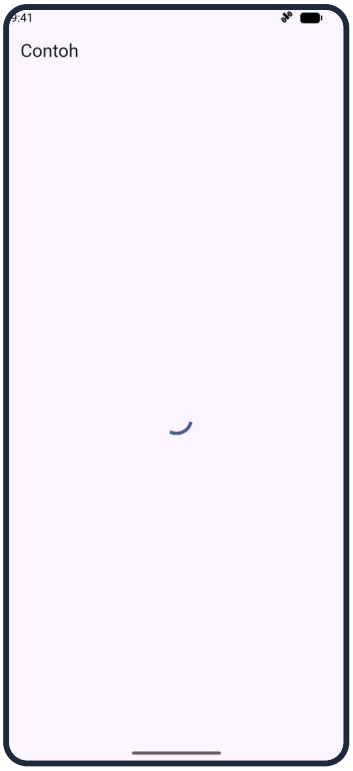
Daftar data jarang langsung tersedia. Data harus dimuat dulu, bisa gagal, atau ternyata kosong. Aplikasi yang baik menampilkan tampilan yang jelas untuk **setiap keadaan**, bukan layar putih kosong.

Konsep: empat keadaan

Keadaan	Yang ditampilkan
Loading	`CircularProgressIndicator` + teks "Memuat..."
Error	Ikon, pesan yang dimengerti pengguna, tombol "Coba Lagi"
Kosong (empty state)	Ikon, penjelasan kenapa kosong, ajakan bertindak
Ada data	ListView / GridView

Contoh kecil: pola operator ternary

```
// Pola umum: periksa keadaan dari yang paling "awal"
return Scaffold(
  body: sedangMemuat
      ? const Center(child: CircularProgressIndicator())       // 1. loading
      : data.isEmpty
          ? const Center(child: Text('Belum ada data'))        // 2. kosong
          : ListView.builder(                                  // 3. ada data
              itemCount: data.length,
              itemBuilder: (context, i) => ListTile(title: Text(data[i])),
            ),
);
```



Keadaan loading: lingkaran berputar di tengah layar

Untuk dua atau tiga keadaan, ternary bersarang masih terbaca. Untuk empat keadaan atau lebih, lebih rapi memakai `enum` + `switch` seperti contoh berikut.

Contoh lengkap: enum StatusMuat

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarTugasScreen(),
    );
  }
}

// Empat kemungkinan keadaan layar daftar
enum StatusMuat { loading, error, kosong, ada }

class DaftarTugasScreen extends StatefulWidget {
```

```dart
  const DaftarTugasScreen({super.key});

  @override
  State<DaftarTugasScreen> createState() => _DaftarTugasScreenState();
}

class _DaftarTugasScreenState extends State<DaftarTugasScreen> {
  StatusMuat _status = StatusMuat.loading;
  List<String> _tugas = [];
  int _percobaan = 0;

  @override
  void initState() {
    super.initState();
    _muatData();
  }

  // Simulasi mengambil data: percobaan ke-1 gagal, ke-2 kosong, ke-3 berhasil
  Future<void> _muatData() async {
    setState(() => _status = StatusMuat.loading);
    await Future.delayed(const Duration(seconds: 2));
    if (!mounted) return;

    _percobaan++;
    setState(() {
      if (_percobaan == 1) {
        _status = StatusMuat.error;
      } else if (_percobaan == 2) {
        _tugas = [];
        _status = StatusMuat.kosong;
      } else {
        _tugas = ['Laporan Praktikum 3', 'Kuis Dart', 'Proyek UI Login'];
        _status = StatusMuat.ada;
      }
    });
  }

  Widget _buildIsi() {
    switch (_status) {
      case StatusMuat.loading:
        return const Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              CircularProgressIndicator(),
              SizedBox(height: 16),
              Text('Memuat tugas...'),
            ],
          ),
        );
      case StatusMuat.error:
        return _PesanStatus(
          ikon: Icons.wifi_off,
          judul: 'Gagal memuat data',
          keterangan: 'Periksa koneksi internet lalu coba lagi.',
          tombol: FilledButton.icon(
            onPressed: _muatData,
            icon: const Icon(Icons.refresh),
            label: const Text('Coba Lagi'),
```

```
          ),
        );
      case StatusMuat.kosong:
        return _PesanStatus(
          ikon: Icons.inbox_outlined,
          judul: 'Belum ada tugas',
          keterangan: 'Tugas dari dosen akan muncul di sini.',
          tombol: OutlinedButton(onPressed: _muatData, child: const Text('Muat Ulang')),
        );
      case StatusMuat.ada:
        return ListView.builder(
          itemCount: _tugas.length,
          itemBuilder: (context, i) => ListTile(
            leading: const Icon(Icons.assignment_outlined),
            title: Text(_tugas[i]),
          ),
        );
    }
  }

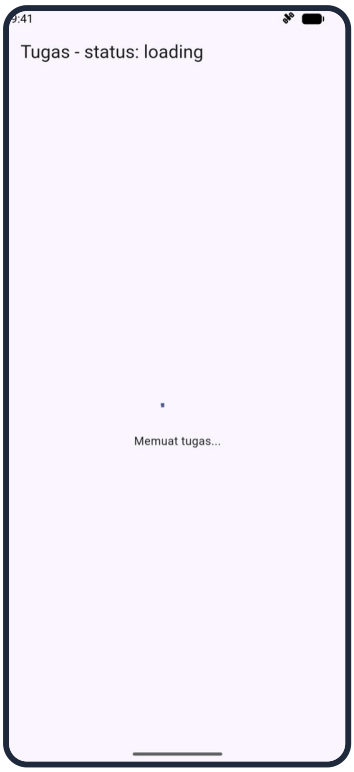
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Tugas - status: ${_status.name}')),
      body: _buildIsi(),
    );
  }
}

// Widget kecil yang bisa dipakai ulang untuk empty state & error state
class _PesanStatus extends StatelessWidget {
  final IconData ikon;
  final String judul;
  final String keterangan;
  final Widget tombol;

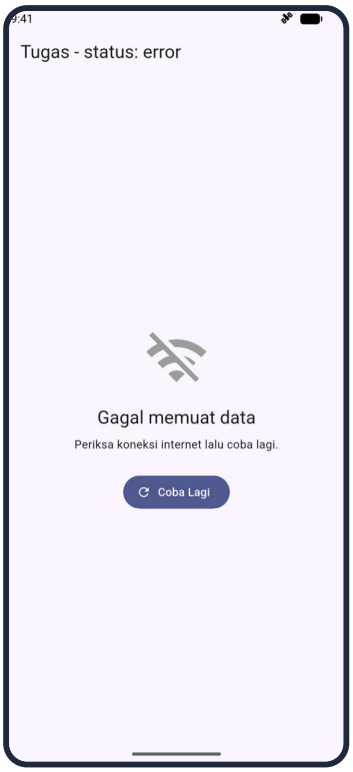
  const _PesanStatus({
    required this.ikon,
    required this.judul,
    required this.keterangan,
    required this.tombol,
  });

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(ikon, size: 80, color: Colors.grey),
            const SizedBox(height: 16),
            Text(judul, style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 8),
            Text(keterangan, textAlign: TextAlign.center),
            const SizedBox(height: 24),
            tombol,
          ],
```

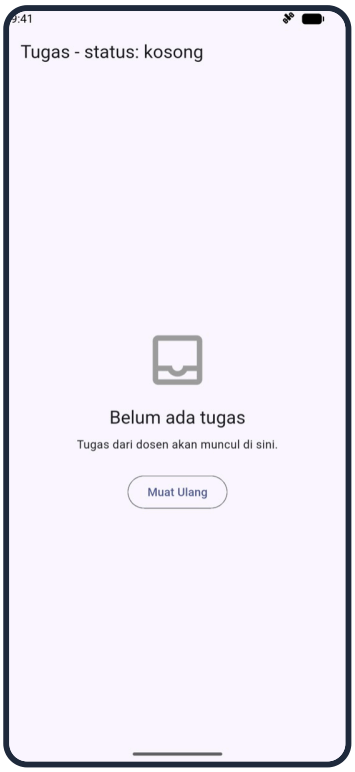
```
        ),
      ),
    );
  }
}
```



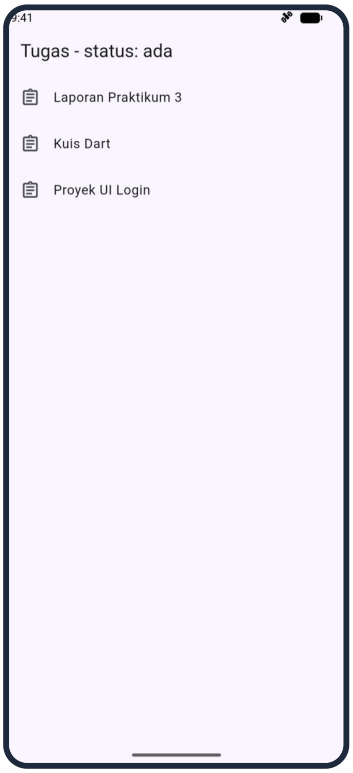
Loading saat pertama dibuka



Percobaan pertama gagal: error state



Setelah Coba Lagi: data kosong



Setelah Muat Ulang: data tampil

Penjelasan kode:

- `enum StatusMuat { loading, error, kosong, ada }` membuat semua keadaan tertulis jelas.
- `switch (_status)` di `_buildIsi` wajib menangani semua nilai enum. Jika kelak ditambah keadaan baru, compiler akan mengingatkan bagian yang belum ditangani.
- Widget `_PesanStatus` dipakai ulang untuk error dan kosong, cukup berbeda ikon, teks, dan tombolnya.
- `_muatData` selalu mengubah status ke loading dulu, lalu menunggu, lalu mengatur hasil. Pola ini juga akan dipakai di Modul 09 saat memanggil API.

> ⚠️ **Kesalahan umum**
>
> - **Hanya memeriksa `list.isEmpty`.** Saat masih loading, list juga kosong, sehingga pengguna sempat melihat tulisan "Belum ada data" padahal data sedang dimuat.
> - **Pesan error teknis** seperti "SocketException: Failed host lookup". Tampilkan bahasa manusia dan berikan tombol coba lagi.
> - **Lupa `if (!mounted) return;`** setelah `await`.

8. Infinite Scroll dengan ScrollController

Aplikasi seperti forum atau media sosial tidak memuat ribuan data sekaligus. Data diambil per halaman (misalnya 15 item), dan halaman berikutnya dimuat saat pengguna mendekati akhir daftar. Teknik ini disebut **infinite scroll** atau *load more*.

Konsep: ScrollController

- `ScrollController` dihubungkan ke ListView lewat `controller:`. Sama seperti TextEditingController: dibuat di State, wajib di-dispose.
- `controller.position.pixels` = jarak yang sudah digulir; `maxScrollExtent` = jarak maksimum.
- `addListener` dipanggil setiap kali posisi gulir berubah.
- `animateTo(0, ...)` / `jumpTo(0)` menggulir ke posisi tertentu, misalnya kembali ke atas.

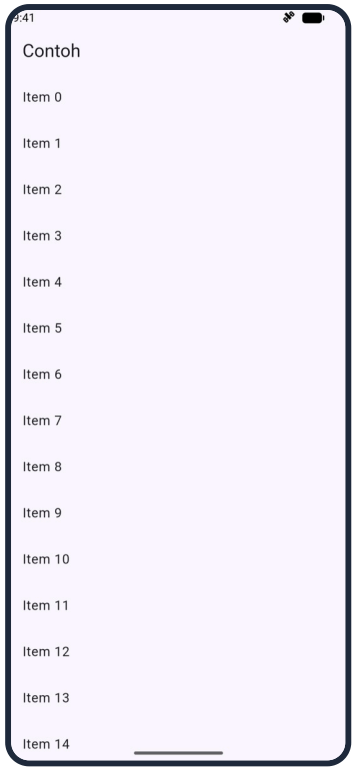
Contoh kecil

```dart
final _scrollController = ScrollController();

@override
void initState() {
  super.initState();
  _scrollController.addListener(() {
    final posisi = _scrollController.position;
    // pixels = jarak yang sudah digulir, maxScrollExtent = jarak maksimum
    if (posisi.pixels >= posisi.maxScrollExtent - 200) {
      debugPrint('Hampir sampai bawah -> muat data berikutnya');
    }
  });
}

@override
void dispose() {
  _scrollController.dispose(); // sama seperti TextEditingController
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return ListView.builder(
    controller: _scrollController, // hubungkan ke list
    itemCount: 50,
    itemBuilder: (context, i) => ListTile(title: Text('Item $i')),
  );
}
```



ListView yang terhubung dengan ScrollController

Contoh lengkap: forum dengan load more

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const ForumScreen(),
    );
  }
}

class ForumScreen extends StatefulWidget {
  const ForumScreen({super.key});

  @override
  State<ForumScreen> createState() => _ForumScreenState();
}

class _ForumScreenState extends State<ForumScreen> {
```

```dart
  static const _ukuranHalaman = 15;
  static const _totalDiServer = 70; // anggap server hanya punya 70 postingan

  final _scrollController = ScrollController();
  final List<String> _postingan = [];
  int _halaman = 0;
  bool _sedangMemuat = false;
  bool _masihAda = true;

  @override
  void initState() {
    super.initState();
    _muatHalamanBerikutnya();
    _scrollController.addListener(_cekPosisiScroll);
  }

  @override
  void dispose() {
    _scrollController.removeListener(_cekPosisiScroll);
    _scrollController.dispose();
    super.dispose();
  }

  void _cekPosisiScroll() {
    final posisi = _scrollController.position;
    if (posisi.pixels >= posisi.maxScrollExtent - 300) {
      _muatHalamanBerikutnya();
    }
  }

  // Simulasi API: GET /postingan?page=N
  Future<List<String>> _ambilDariServer(int halaman) async {
    await Future.delayed(const Duration(milliseconds: 1200));
    final mulai = halaman * _ukuranHalaman;
    final akhir = (mulai + _ukuranHalaman).clamp(0, _totalDiServer);
    return [for (var i = mulai; i < akhir; i++) 'Postingan forum #${i + 1}'];
  }

  Future<void> _muatHalamanBerikutnya() async {
    // Penjaga: jangan memuat dua kali bersamaan & berhenti jika data habis
    if (_sedangMemuat || !_masihAda) return;
    setState(() => _sedangMemuat = true);

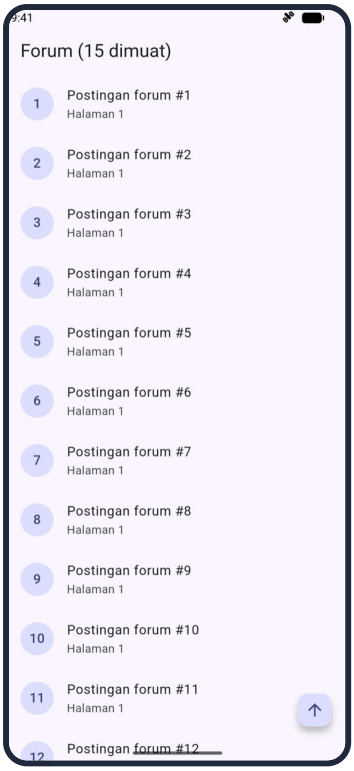
    final baru = await _ambilDariServer(_halaman);
    if (!mounted) return;

    setState(() {
      _postingan.addAll(baru);
      _halaman++;
      _sedangMemuat = false;
      _masihAda = baru.length == _ukuranHalaman;
    });
  }

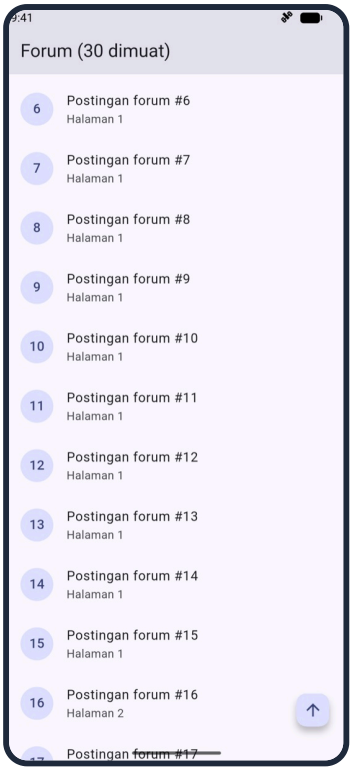
  Future<void> _refresh() async {
    setState(() {
      _postingan.clear();
      _halaman = 0;
      _masihAda = true;
```

```
    });
    await _muatHalamanBerikutnya();
  }

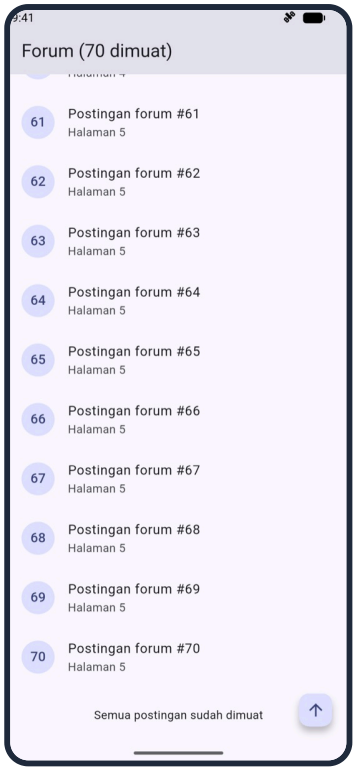
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Forum (${_postingan.length} dimuat)')),
      body: RefreshIndicator(
        onRefresh: _refresh,
        child: ListView.builder(
          controller: _scrollController,
          physics: const AlwaysScrollableScrollPhysics(),
          // +1 untuk baris paling bawah (loading / "sudah habis")
          itemCount: _postingan.length + 1,
          itemBuilder: (context, index) {
            if (index == _postingan.length) {
              return Padding(
                padding: const EdgeInsets.all(24),
                child: Center(
                  child: _masihAda
                      ? const CircularProgressIndicator()
                      : const Text('Semua postingan sudah dimuat'),
                ),
              );
            }
            return ListTile(
              leading: CircleAvatar(child: Text('${index + 1}')),
              title: Text(_postingan[index]),
              subtitle: Text('Halaman ${index ~/ _ukuranHalaman + 1}'),
            );
          },
        ),
      ),
      floatingActionButton: FloatingActionButton.small(
        tooltip: 'Ke atas',
        // animateTo: gulir ke posisi tertentu dengan animasi
        onPressed: () => _scrollController.animateTo(
          0,
          duration: const Duration(milliseconds: 500),
          curve: Curves.easeOut,
        ),
        child: const Icon(Icons.arrow_upward),
      ),
    );
  }
}
```



15 postingan pertama dimuat



Setelah digulir ke bawah: halaman kedua otomatis dimuat (30 postingan)



Semua 70 postingan sudah dimuat

Penjelasan kode:

- `_cekPosisiScroll` memuat halaman berikutnya saat posisi tinggal 300 piksel dari bawah, sehingga data baru sudah siap sebelum pengguna benar-benar sampai di ujung.
- Dua penjaga di awal `_muatHalamanBerikutnya` sangat penting: `_sedangMemuat` mencegah permintaan ganda (listener terpanggil puluhan kali per detik), dan `_masihAda` menghentikan pemuatan jika server sudah tidak punya data.
- `itemCount: _postingan.length + 1` : item terakhir dipakai sebagai baris status (loading atau "sudah habis").
- Data dianggap habis jika halaman yang diterima kurang dari `_ukuranHalaman`.
- FAB memakai `animateTo(0)` untuk kembali ke atas, dan `_refresh` mereset semuanya ke halaman pertama.

> ⚠️ **Kesalahan umum**
>
> - **Tanpa penjaga `_sedangMemuat` :** halaman yang sama dimuat berkali-kali sehingga data dobel.
> - **Lupa `dispose` ScrollController dan `removeListener` .**
> - **Membaca `controller.position` sebelum ListView tampil.** Controller belum terpasang; periksa dengan `controller.hasClients` jika perlu.

9. Pencarian & Filter Daftar

Fitur pencarian membuat daftar panjang mudah dipakai. Kuncinya sederhana: **jangan ubah data asli**. Simpan kata kunci di State, lalu hitung daftar yang ditampilkan dari data asli setiap kali `build`.

Konsep

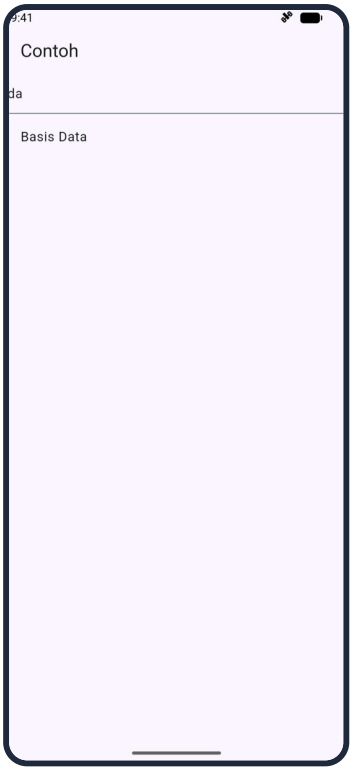
1. `_semua`: data asli, tidak pernah dikurangi oleh pencarian.
2. `_kataKunci`: diperbarui oleh `onChanged` TextField + `setState`.
3. Daftar tampil = `_semua.where(...)` berdasarkan kata kunci.

Contoh kecil

```
final List<String> _semua = ['Algoritma', 'Basis Data', 'Kalkulus', 'Jaringan'];
String _kataKunci = '';

@override
Widget build(BuildContext context) {
  // Data yang DITAMPILKAN dihitung ulang setiap build dari data asli
  final hasil = _semua
      .where((m) => m.toLowerCase().contains(_kataKunci.toLowerCase()))
      .toList();

  return Column(
    children: [
      TextField(
        decoration: const InputDecoration(hintText: 'Cari...'),
        onChanged: (teks) => setState(() => _kataKunci = teks),
      ),
      Expanded(
        child: ListView.builder(
          itemCount: hasil.length,
          itemBuilder: (context, i) => ListTile(title: Text(hasil[i])),
        ),
      ),
    ],
  );
}
```



Mengetik "da" menyisakan Basis Data

`toLowerCase()` pada kedua sisi membuat pencarian tidak membedakan huruf besar dan kecil.

Contoh lengkap: direktori dosen dengan pencarian dan filter

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const DirektoriDosenScreen(),
    );
  }
}

class Dosen {
  final String nama;
  final String bidang;
  final bool tersedia;

  const Dosen(this.nama, this.bidang, this.tersedia);
```

```dart
}

class DirektoriDosenScreen extends StatefulWidget {
  const DirektoriDosenScreen({super.key});

  @override
  State<DirektoriDosenScreen> createState() => _DirektoriDosenScreenState();
}

class _DirektoriDosenScreenState extends State<DirektoriDosenScreen> {
  static const _semuaDosen = [
    Dosen('Dr. Bambang Susilo', 'Kecerdasan Buatan', true),
    Dosen('Nadia Fitri, M.T.', 'Rekayasa Perangkat Lunak', false),
    Dosen('Irham Akbar, M.Cs.', 'Pemrograman Mobile', true),
    Dosen('Prof. Hartono', 'Kecerdasan Buatan', false),
    Dosen('Eko Prasetyo, Ph.D.', 'Keamanan Siber', true),
    Dosen('Sari Wulandari, M.Kom.', 'Basis Data', true),
    Dosen('Andi Wijaya, M.T.', 'Jaringan Komputer', false),
    Dosen('Rina Kartika, M.Sc.', 'Data Science', true),
  ];

  final _cariController = TextEditingController();
  String _kataKunci = '';
  String _bidang = 'Semua';
  bool _hanyaTersedia = false;

  @override
  void dispose() {
    _cariController.dispose();
    super.dispose();
  }

  List<Dosen> get _hasilFilter {
    final kunci = _kataKunci.toLowerCase().trim();
    return _semuaDosen.where((d) {
      final cocokTeks = d.nama.toLowerCase().contains(kunci) ||
          d.bidang.toLowerCase().contains(kunci);
      final cocokBidang = _bidang == 'Semua' || d.bidang == _bidang;
      final cocokStatus = !_hanyaTersedia || d.tersedia;
      return cocokTeks && cocokBidang && cocokStatus;
    }).toList();
  }

  @override
  Widget build(BuildContext context) {
    final hasil = _hasilFilter;
    final daftarBidang = ['Semua', ..._semuaDosen.map((d) => d.bidang).toSet()];

    return Scaffold(
      appBar: AppBar(title: const Text('Direktori Dosen')),
      body: Column(
        children: [
          // 1. Kolom pencarian
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 0),
            child: TextField(
              controller: _cariController,
              decoration: InputDecoration(
                hintText: 'Cari nama atau bidang...',
```

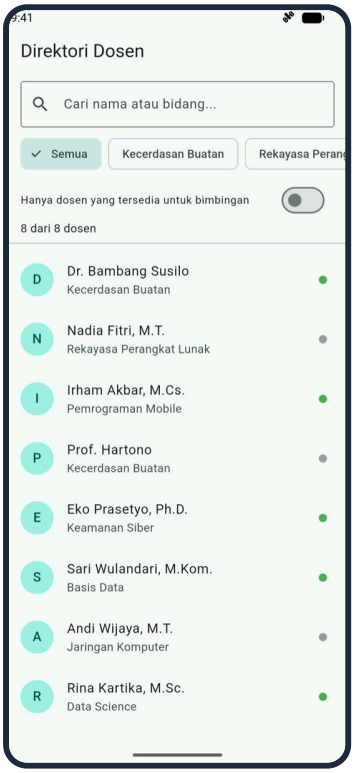
```
            prefixIcon: const Icon(Icons.search),
            border: const OutlineInputBorder(),
            suffixIcon: _kataKunci.isEmpty
                ? null
                : IconButton(
                    icon: const Icon(Icons.clear),
                    onPressed: () {
                      _cariController.clear();
                      setState(() => _kataKunci = '');
                    },
                  ),
          ),
          onChanged: (teks) => setState(() => _kataKunci = teks),
        ),
      ),

      // 2. Filter bidang dengan ChoiceChip (gulir horizontal)
      SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
        child: Row(
          children: [
            for (final b in daftarBidang)
              Padding(
                padding: const EdgeInsets.symmetric(horizontal: 4),
                child: ChoiceChip(
                  label: Text(b),
                  selected: _bidang == b,
                  onSelected: (_) => setState(() => _bidang = b),
                ),
              ),
          ],
        ),
      ),

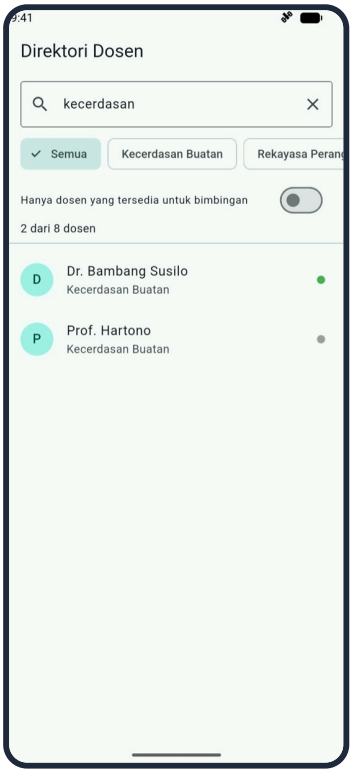
      // 3. Filter status
      SwitchListTile(
        dense: true,
        title: const Text('Hanya dosen yang tersedia untuk bimbingan'),
        value: _hanyaTersedia,
        onChanged: (v) => setState(() => _hanyaTersedia = v),
      ),
      Padding(
        padding: const EdgeInsets.symmetric(horizontal: 16),
        child: Align(
          alignment: Alignment.centerLeft,
          child: Text('${hasil.length} dari ${_semuaDosen.length} dosen'),
        ),
      ),
      const Divider(),

      // 4. Hasil (Expanded agar ListView punya tinggi yang jelas)
      Expanded(
        child: hasil.isEmpty
            ? const Center(child: Text('Tidak ada dosen yang cocok'))
            : ListView.builder(
                itemCount: hasil.length,
                itemBuilder: (context, i) {
                  final d = hasil[i];
```

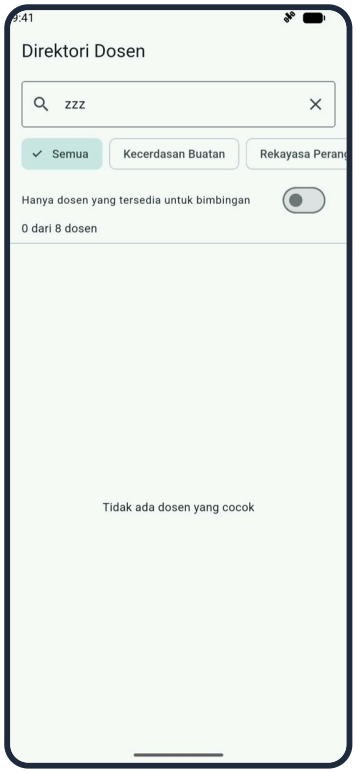
```
                    return ListTile(
                      leading: CircleAvatar(child: Text(d.nama[0])),
                      title: Text(d.nama),
                      subtitle: Text(d.bidang),
                      trailing: Icon(
                        Icons.circle,
                        size: 12,
                        color: d.tersedia ? Colors.green : Colors.grey,
                      ),
                    );
                  },
                ),
            ),
          ],
        ),
      );
    }
}
```



Direktori dosen tanpa filter



Mengetik "kecerdasan" di kolom pencarian



Kata kunci tanpa hasil: pesan kosong

Penjelasan kode:

- Getter `_hasilFilter` menggabungkan tiga syarat dengan `&&`: cocok teks (nama atau bidang), cocok bidang dari chip, dan cocok status ketersediaan.
- `.map((d) => d.bidang).toSet()` menghilangkan bidang yang kembar sebelum dibuat menjadi chip. `['Semua', ...]` menambahkan pilihan "Semua" di depan dengan *spread operator*.
- `ChoiceChip` adalah chip yang hanya bisa dipilih satu (seperti radio).
- Daftar dibungkus `Expanded` karena berada di dalam `Column`.
- Teks "x dari y dosen" memberi tahu pengguna bahwa filter sedang aktif.

⚠️ **Kesalahan umum**

- **Menghapus item dari data asli saat mencari** (`_semua.removeWhere(...)`). Setelah kata kunci dihapus, data tidak bisa kembali.
- **Memakai index hasil filter untuk menghapus dari data asli.** Index keduanya berbeda; hapus berdasarkan objek atau id (lihat studi kasus).
- **Lupa** `setState` di `onChanged`, sehingga daftar tidak berubah saat mengetik.

10. CustomScrollView, SliverAppBar, SliverList & SliverGrid

Sliver adalah potongan area gulir. Dengan `CustomScrollView`, beberapa sliver (app bar, grid, list, widget biasa) digabung menjadi **satu** area gulir yang mulus. Contoh paling terkenal: header bergambar yang mengecil saat digulir (*collapsing header*).

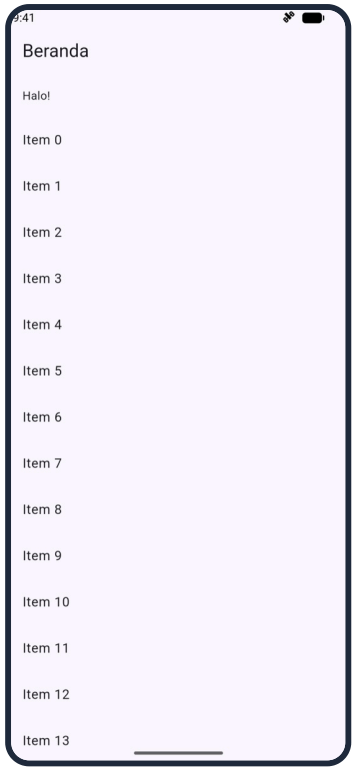
Konsep

Sliver	Padanan widget biasa
`SliverAppBar`	`AppBar` yang bisa membesar/mengecil
`SliverList.builder` / `.separated`	`ListView.builder` / `.separated`
`SliverGrid.count` / `SliverGrid.builder`	`GridView.count` / `.builder`
`SliverToBoxAdapter`	Pembungkus untuk widget biasa (Text, Card, dll.)
`SliverPadding`	`Padding` untuk sliver

Properti penting `SliverAppBar`: `expandedHeight` (tinggi saat terbuka penuh), `pinned` (bagian kecilnya tetap menempel di atas), `floating` (muncul lagi begitu pengguna menggulir sedikit ke atas), `snap` (bersama floating, langsung terbuka penuh), dan `flexibleSpace` (isi area yang membesar).

Contoh kecil

```
body: CustomScrollView(
  // Semua anak CustomScrollView harus berupa SLIVER
  slivers: [
    const SliverAppBar(title: Text('Beranda'), pinned: true),
    // Widget biasa dibungkus SliverToBoxAdapter
    const SliverToBoxAdapter(
      child: Padding(padding: EdgeInsets.all(16), child: Text('Halo!')),
    ),
    SliverList.builder(
      itemCount: 30,
      itemBuilder: (context, i) => ListTile(title: Text('Item $i')),
    ),
  ],
),
```



CustomScrollView dengan SliverAppBar, teks, dan SliverList

Contoh lengkap 1: profil universitas dengan gambar header

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

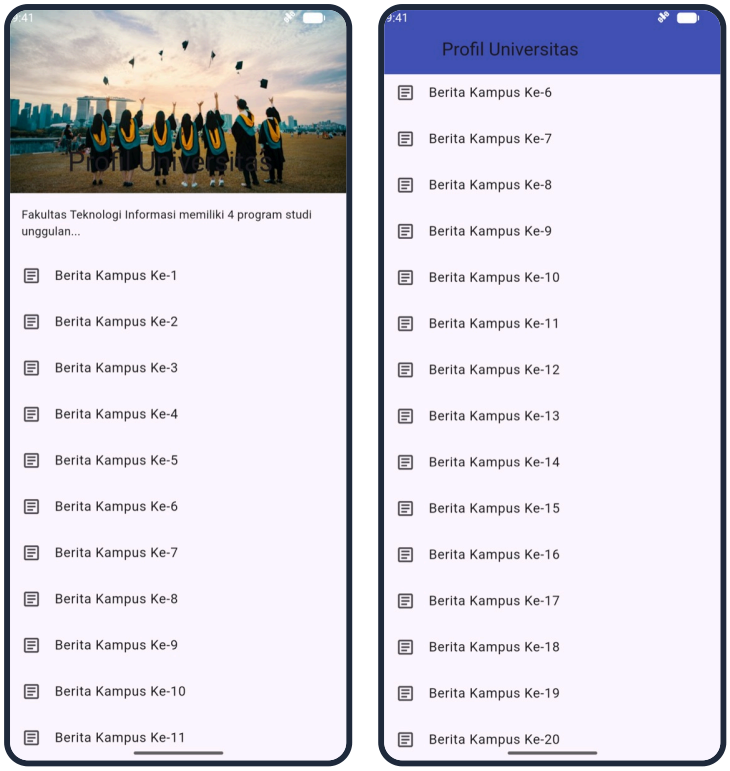
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: ProfilUniversitasScreen(),
    );
  }
}

class ProfilUniversitasScreen extends StatelessWidget {
  const ProfilUniversitasScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // CustomScrollView TIDAK memakai AppBar biasa: SliverAppBar sudah menjadi app bar-nya
    return Scaffold(
      body: CustomScrollView(
        slivers: [
```

```
          SliverAppBar(
            expandedHeight: 200.0,
            floating: false,
            pinned: true, // Judul tetap menempel di atas saat discroll
            backgroundColor: Colors.indigo,
            foregroundColor: Colors.white,
            flexibleSpace: FlexibleSpaceBar(
              title: const Text("Profil Universitas"),
              background: Image.network(
                "https://images.unsplash.com/photo-1541339907198-e08756dedf3f?w=600",
                fit: BoxFit.cover,
                // Cadangan jika gambar gagal dimuat (offline / URL mati)
                errorBuilder: (context, error, stackTrace) =>
                    Container(color: Colors.indigo.shade300),
              ),
            ),
          ),
          const SliverToBoxAdapter(
            child: Padding(
              padding: EdgeInsets.all(16),
              child: Text("Fakultas Teknologi Informasi memiliki 4 program studi unggulan..."),
            ),
          ),
          SliverList.builder(
            itemCount: 20,
            itemBuilder: (context, index) => ListTile(
              leading: const Icon(Icons.article_outlined),
              title: Text("Berita Kampus Ke-${index + 1}"),
            ),
          ),
        ],
      ),
    );
  }
}
```



Header gambar terbuka penuh

Setelah digulir: header mengecil menjadi app bar biasa

- `FlexibleSpaceBar` mengatur judul yang mengecil dan gambar latar yang memudar.
- `errorBuilder` pada `Image.network` menampilkan warna polos jika gambar gagal dimuat (misalnya saat offline).

Contoh lengkap 2: dashboard dengan grid dan list dalam satu gulir

Tanpa sliver, menaruh GridView dan ListView dalam satu halaman yang bisa digulir bersama cukup merepotkan. Dengan `CustomScrollView`, keduanya cukup ditumpuk sebagai sliver.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DashboardScreen(),
    );
  }
}
```

```dart
class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  static const _menu = [
    (Icons.edit_note, 'KRS'),
    (Icons.grading, 'KHS'),
    (Icons.calendar_month, 'Jadwal'),
    (Icons.fact_check, 'Presensi'),
    (Icons.receipt_long, 'Tagihan'),
    (Icons.local_library, 'Pustaka'),
  ];

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;

    return Scaffold(
      body: CustomScrollView(
        slivers: [
          // 1. App bar yang mengecil saat digulir
          SliverAppBar(
            expandedHeight: 180,
            pinned: true, // bagian kecilnya tetap menempel di atas
            backgroundColor: warna.primary,
            foregroundColor: warna.onPrimary,
            flexibleSpace: FlexibleSpaceBar(
              title: const Text('Halo, Nadia'),
              background: Container(
                decoration: BoxDecoration(
                  gradient: LinearGradient(
                    colors: [warna.primary, warna.tertiary],
                    begin: Alignment.topLeft,
                    end: Alignment.bottomRight,
                  ),
                ),
                alignment: Alignment.centerRight,
                padding: const EdgeInsets.only(right: 24),
                child: Icon(Icons.school, size: 96, color: warna.onPrimary.withValues(alpha: 0.3)),
              ),
            ),
          ),

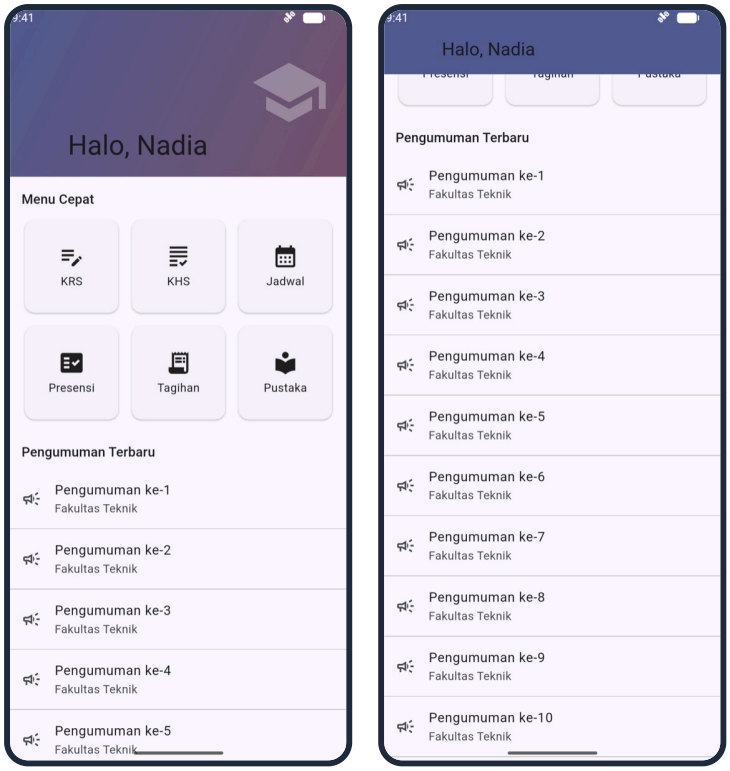
          // 2. Widget biasa di antara sliver
          SliverToBoxAdapter(
            child: Padding(
              padding: const EdgeInsets.fromLTRB(16, 16, 16, 8),
              child: Text('Menu Cepat', style: Theme.of(context).textTheme.titleMedium),
            ),
          ),

          // 3. Grid di dalam CustomScrollView
          SliverPadding(
            padding: const EdgeInsets.symmetric(horizontal: 16),
            sliver: SliverGrid.count(
              crossAxisCount: 3,
              mainAxisSpacing: 8,
              crossAxisSpacing: 8,
              children: [
```

```
                for (final (ikon, label) in _menu)
                  Card(
                    child: Column(
                      mainAxisAlignment: MainAxisAlignment.center,
                      children: [Icon(ikon, size: 32), const SizedBox(height: 4), Text(label)],
                    ),
                  ),
              ],
            ),
          ),

          SliverToBoxAdapter(
            child: Padding(
              padding: const EdgeInsets.fromLTRB(16, 24, 16, 8),
              child: Text('Pengumuman Terbaru', style: Theme.of(context).textTheme.titleMedium),
            ),
          ),

          // 4. Daftar panjang di bawahnya: satu scroll untuk semuanya
          SliverList.separated(
            itemCount: 15,
            itemBuilder: (context, i) => ListTile(
              leading: const Icon(Icons.campaign_outlined),
              title: Text('Pengumuman ke-${i + 1}'),
              subtitle: const Text('Fakultas Teknik'),
            ),
            separatorBuilder: (context, i) => const Divider(height: 1),
          ),
        ],
      ),
    );
  }
}
```



Dashboard: header gradasi, grid menu, dan daftar pengumuman

Setelah digulir: header menempel, list berlanjut

Penjelasan kode:

- Header memakai gradasi warna dari `colorScheme`, sehingga ikut berubah jika `colorSchemeSeed` diganti.
- `SliverPadding` memberi jarak kiri-kanan untuk `SliverGrid.count`. `Padding` biasa tidak boleh membungkus sliver.
- Judul bagian ("Menu Cepat", "Pengumuman Terbaru") adalah widget biasa sehingga dibungkus `SliverToBoxAdapter`.
- `SliverList.separated` bekerja persis seperti `ListView.separated`, termasuk lazy loading.

> ⚠️ **Kesalahan umum**
>
> - **Memasukkan widget biasa langsung ke `slivers`**, misalnya `Text` atau `ListView`. Muncul error "A RenderViewport expected a child of type RenderSliver". Bungkus dengan `SliverToBoxAdapter` atau ganti dengan versi sliver-nya.
> - **Memakai `appBar:` Scaffold sekaligus SliverAppBar.** Hasilnya dua app bar bertumpuk.
> - **Memakai `snap: true` tanpa `floating: true`.** Terjadi error assertion; snap hanya berlaku bersama floating.

11. Studi Kasus: Daftar Mata Kuliah

Studi kasus ini menggabungkan materi Modul 05 dan 06 menjadi aplikasi **Daftar Mata Kuliah** dengan fitur: pencarian, hapus dengan geser + urungkan, tambah lewat form di bottom sheet (dengan validasi), ringkasan total SKS, dan empty state.

Rancangan

- **Model** `MataKuliah` (kode, nama, sks, semester). `kode` dipakai sebagai id unik untuk `ValueKey` Dismissible.
- **Data asli** `_semua` dan **data tampil** `_tampil` (hasil filter + urut semester) dipisah.
- **Form tambah** dibuat sebagai StatefulWidget terpisah (`FormMatkulSheet`) agar controller-nya punya `dispose` sendiri. Form mengembalikan objek `MataKuliah` lewat `Navigator.pop`.

lib/main.dart

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Mata Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const DaftarMatkulScreen(),
    );
  }
}

// ======================= MODEL =======================
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;
  final int semester;

  const MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.semester,
  });
}

// ======================= LAYAR UTAMA =======================
class DaftarMatkulScreen extends StatefulWidget {
```

```dart
  const DaftarMatkulScreen({super.key});

  @override
  State<DaftarMatkulScreen> createState() => _DaftarMatkulScreenState();
}

class _DaftarMatkulScreenState extends State<DaftarMatkulScreen> {
  final List<MataKuliah> _semua = [
    const MataKuliah(kode: 'IF101', nama: 'Algoritma dan Pemrograman', sks: 3, semester: 1),
    const MataKuliah(kode: 'IF102', nama: 'Matematika Diskrit', sks: 3, semester: 1),
    const MataKuliah(kode: 'IF201', nama: 'Struktur Data', sks: 3, semester: 2),
    const MataKuliah(kode: 'IF202', nama: 'Basis Data', sks: 3, semester: 3),
    const MataKuliah(kode: 'IF305', nama: 'Pemrograman Mobile', sks: 4, semester: 5),
    const MataKuliah(kode: 'IF306', nama: 'Jaringan Komputer', sks: 3, semester: 5),
    const MataKuliah(kode: 'IF401', nama: 'Kecerdasan Buatan', sks: 3, semester: 6),
  ];

  final _cariController = TextEditingController();
  String _kataKunci = '';

  @override
  void dispose() {
    _cariController.dispose();
    super.dispose();
  }

  List<MataKuliah> get _tampil {
    final k = _kataKunci.toLowerCase().trim();
    final hasil = _semua
        .where((m) => m.nama.toLowerCase().contains(k) || m.kode.toLowerCase().contains(k))
        .toList();
    hasil.sort((a, b) => a.semester.compareTo(b.semester)); // urut per semester
    return hasil;
  }

  int get _totalSks => _semua.fold(0, (total, m) => total + m.sks);

  // ---------- HAPUS + URUNGKAN ----------
  void _hapus(MataKuliah m) {
    // Simpan posisi di list ASLI (bukan list hasil filter)
    final posisi = _semua.indexOf(m);
    setState(() => _semua.removeAt(posisi));

    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          content: Text('${m.nama} dihapus'),
          action: SnackBarAction(
            label: 'URUNGKAN',
            onPressed: () => setState(() => _semua.insert(posisi, m)),
          ),
        ),
      );
  }

  // ---------- TAMBAH lewat bottom sheet ----------
  Future<void> _bukaFormTambah() async {
    final baru = await showModalBottomSheet<MataKuliah>(
```

```
      context: context,
      isScrollControlled: true, // agar sheet bisa naik saat keyboard muncul
      showDragHandle: true,
      builder: (_) => FormMatkulSheet(kodeTerpakai: _semua.map((m) => m.kode).toSet()),
    );
    if (baru == null || !mounted) return;

    setState(() => _semua.add(baru));
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('${baru.nama} ditambahkan')),
    );
  }

  @override
  Widget build(BuildContext context) {
    final tampil = _tampil;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Mata Kuliah Saya'),
        bottom: PreferredSize(
          preferredSize: const Size.fromHeight(72),
          child: Padding(
            padding: const EdgeInsets.fromLTRB(16, 0, 16, 12),
            child: SearchBar(
              controller: _cariController,
              hintText: 'Cari nama atau kode...',
              leading: const Icon(Icons.search),
              trailing: [
                if (_kataKunci.isNotEmpty)
                  IconButton(
                    icon: const Icon(Icons.clear),
                    onPressed: () {
                      _cariController.clear();
                      setState(() => _kataKunci = '');
                    },
                  ),
              ],
              onChanged: (v) => setState(() => _kataKunci = v),
            ),
          ),
        ),
      ),
      body: Column(
        children: [
          // Ringkasan
          ListTile(
            tileColor: Theme.of(context).colorScheme.surfaceContainerHighest,
            leading: const Icon(Icons.summarize_outlined),
            title: Text('${_semua.length} mata kuliah - $_totalSks SKS'),
            subtitle: Text(_kataKunci.isEmpty
                ? 'Geser ke kiri untuk menghapus'
                : '${tampil.length} hasil untuk "$_kataKunci"'),
          ),
          Expanded(child: _buildDaftar(tampil)),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _bukaFormTambah,
```

```
        icon: const Icon(Icons.add),
        label: const Text('Tambah'),
      ),
    );
  }

  Widget _buildDaftar(List<MataKuliah> tampil) {
    // Empty state: dua kemungkinan berbeda
    if (_semua.isEmpty) {
      return const _Kosong(ikon: Icons.inbox_outlined, pesan: 'Belum ada mata kuliah.\nTekan
Tambah.');
    }
    if (tampil.isEmpty) {
      return const _Kosong(ikon: Icons.search_off, pesan: 'Tidak ada yang cocok.');
    }

    return ListView.builder(
      padding: const EdgeInsets.only(bottom: 88), // ruang untuk FAB
      itemCount: tampil.length,
      itemBuilder: (context, i) {
        final m = tampil[i];
        return Dismissible(
          key: ValueKey(m.kode), // kode unik
          direction: DismissDirection.endToStart,
          background: Container(
            color: Colors.red,
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 24),
            child: const Icon(Icons.delete, color: Colors.white),
          ),
          onDismissed: (_) => _hapus(m),
          child: ListTile(
            leading: CircleAvatar(child: Text('${m.sks}')),
            title: Text(m.nama),
            subtitle: Text('${m.kode} - Semester ${m.semester}'),
            trailing: const Text('SKS'),
          ),
        );
      },
    );
  }
}

class _Kosong extends StatelessWidget {
  final IconData ikon;
  final String pesan;

  const _Kosong({required this.ikon, required this.pesan});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(ikon, size: 72, color: Colors.grey),
          const SizedBox(height: 12),
          Text(pesan, textAlign: TextAlign.center),
        ],
```

```
      ),
    );
  }
}

// ======================== FORM TAMBAH (di dalam bottom sheet) ========================
class FormMatkulSheet extends StatefulWidget {
  final Set<String> kodeTerpakai;

  const FormMatkulSheet({super.key, required this.kodeTerpakai});

  @override
  State<FormMatkulSheet> createState() => _FormMatkulSheetState();
}

class _FormMatkulSheetState extends State<FormMatkulSheet> {
  final _formKey = GlobalKey<FormState>();
  final _kodeC = TextEditingController();
  final _namaC = TextEditingController();
  int? _sks;
  double _semester = 1;

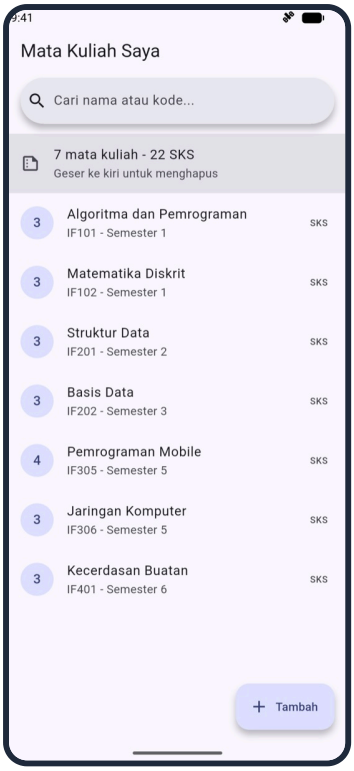
  @override
  void dispose() {
    _kodeC.dispose();
    _namaC.dispose();
    super.dispose();
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    Navigator.pop(
      context,
      MataKuliah(
        kode: _kodeC.text.trim().toUpperCase(),
        nama: _namaC.text.trim(),
        sks: _sks!,
        semester: _semester.round(),
      ),
    );
  }

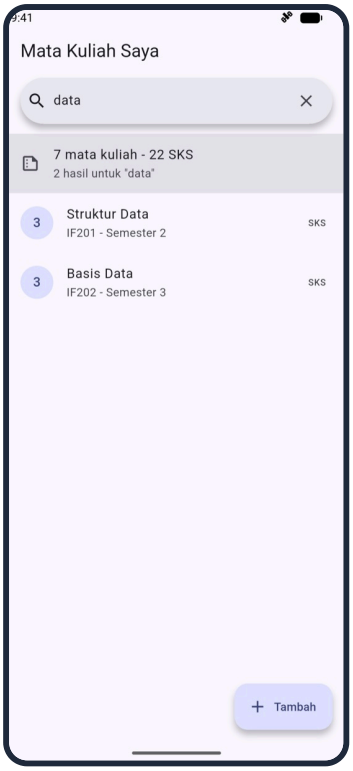
  @override
  Widget build(BuildContext context) {
    // viewInsets.bottom = tinggi keyboard; padding ini mendorong form ke atas
    final tinggiKeyboard = MediaQuery.viewInsetsOf(context).bottom;

    return Padding(
      padding: EdgeInsets.fromLTRB(24, 0, 24, 24 + tinggiKeyboard),
      child: Form(
        key: _formKey,
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('Tambah Mata Kuliah', style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 16),
            TextFormField(
              controller: _kodeC,
```

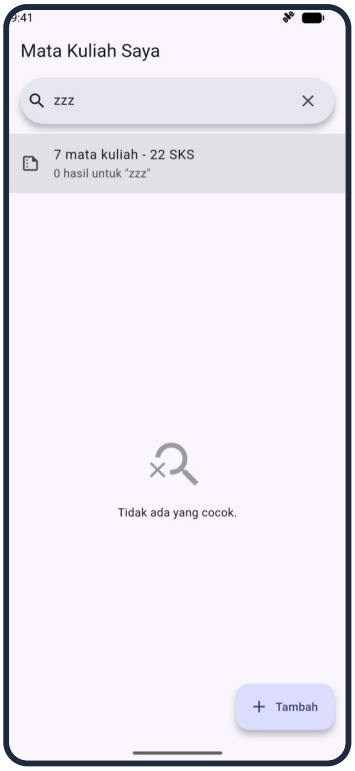
```
              autofocus: true,
              textCapitalization: TextCapitalization.characters,
              textInputAction: TextInputAction.next,
              decoration: const InputDecoration(
                labelText: 'Kode (contoh: IF307)',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                final kode = (v ?? '').trim().toUpperCase();
                if (kode.isEmpty) return 'Kode wajib diisi';
                if (!RegExp(r'^[A-Z]{2}\d{3}$').hasMatch(kode)) return 'Format: 2 huruf + 3 angka';
                if (widget.kodeTerpakai.contains(kode)) return 'Kode sudah ada';
                return null;
              },
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _namaC,
              textCapitalization: TextCapitalization.words,
              decoration: const InputDecoration(
                labelText: 'Nama Mata Kuliah',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                  (v == null || v.trim().length < 3) ? 'Nama minimal 3 karakter' : null,
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<int>(
              initialValue: _sks,
              decoration: const InputDecoration(labelText: 'SKS', border: OutlineInputBorder()),
              items: [for (final s in [1, 2, 3, 4, 6]) DropdownMenuItem(value: s, child: Text('$s
SKS'))],
              onChanged: (v) => setState(() => _sks = v),
              validator: (v) => v == null ? 'Pilih jumlah SKS' : null,
            ),
            const SizedBox(height: 12),
            Text('Semester: ${_semester.round()}'),
            Slider(
              value: _semester,
              min: 1,
              max: 8,
              divisions: 7,
              label: '${_semester.round()}',
              onChanged: (v) => setState(() => _semester = v),
            ),
            FilledButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```



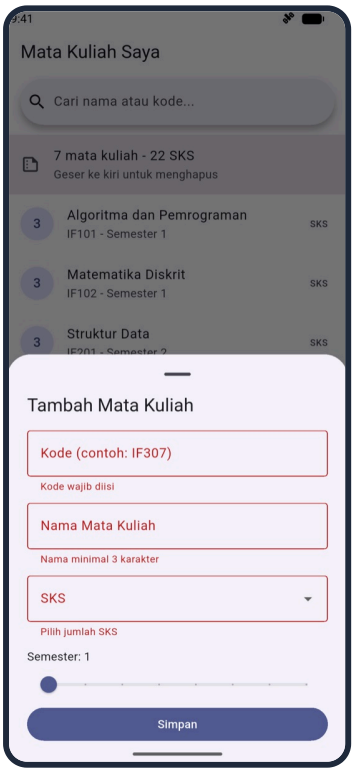
Daftar mata kuliah dengan ringkasan SKS



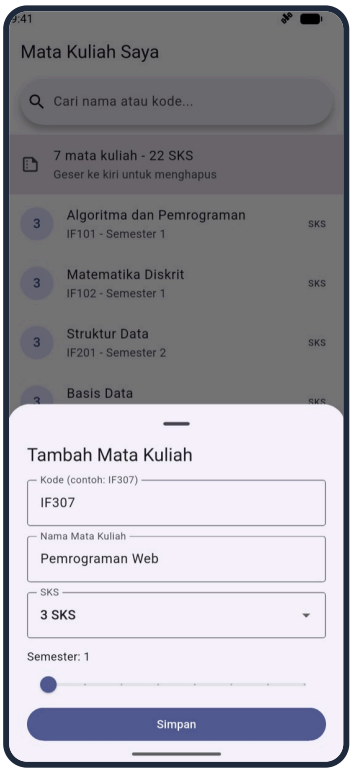
Mencari "data": dua hasil



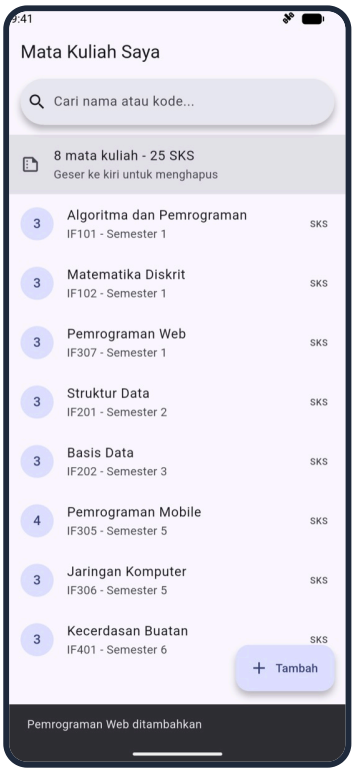
Pencarian tanpa hasil



Form tambah di bottom sheet: Simpan ditekan saat kosong



Form tambah setelah diisi



Mata kuliah baru masuk ke daftar

Penjelasan bagian penting:

- **Hapus dari hasil filter:** `_hapus(m)` menerima objek, lalu mencari posisinya di `_semua` dengan `indexOf`. Dengan begitu penghapusan tetap benar walaupun daftar sedang difilter atau diurutkan.

- **SearchBar** adalah kolom pencarian bergaya Material 3. Diletakkan di `AppBar.bottom` melalui `PreferredSize`.
- **Dua jenis empty state:** "belum ada mata kuliah" (data asli kosong) berbeda dengan "tidak ada yang cocok" (filter tanpa hasil).
- **Bottom sheet berisi form:** `isScrollControlled: true` membolehkan sheet lebih tinggi dari setengah layar, dan padding bawah sebesar `MediaQuery.viewInsetsOf(context).bottom` (tinggi keyboard) mendorong form ke atas keyboard.
- **Validasi kode unik:** daftar kode yang sudah dipakai dikirim ke form sebagai `Set<String>`, lalu diperiksa di validator.
- **Total SKS** dihitung dengan `fold`: mulai dari 0, tambahkan sks setiap mata kuliah.
- `padding: EdgeInsets.only(bottom: 88)` pada ListView memberi ruang agar item terakhir tidak tertutup FAB.

> 💡 **Pengembangan lanjutan**
>
> - Tambahkan fitur edit: ketuk item untuk membuka `FormMatkulSheet` berisi data lama.
> - Kelompokkan daftar per semester dengan judul bagian.
> - Data saat ini hilang saat aplikasi ditutup. Pada Modul 10-12 data akan disimpan ke penyimpanan lokal dan Supabase.

Tugas Praktikum: Praktikum 6: Direktori Dosen dengan Pencarian, Swipe, dan Load More

Sasaran Praktikum

Membangun direktori dosen interaktif yang tetap lancar dengan banyak data, lengkap dengan pencarian, filter, swipe, pull-to-refresh, dan penanganan setiap keadaan layar.

Langkah Pengerjaan

1. Buat model `Dosen` (id, nama, NIDN, bidang keahlian, tersedia untuk bimbingan) dan dataset minimal 40 data (boleh dibuat dengan `List.generate`).
2. Tampilkan data dengan `ListView.builder` atau `ListView.separated`; setiap baris berupa `ListTile` dengan avatar inisial, nama, bidang, dan indikator ketersediaan.
3. Tambahkan `TextField` atau `SearchBar` untuk mencari berdasarkan nama atau bidang tanpa mengubah data asli, plus `ChoiceChip` untuk filter bidang.
4. Tambahkan `Dismissible`: geser ke kanan untuk menandai favorit (item tidak hilang), geser ke kiri untuk menghapus dengan dialog konfirmasi dan SnackBar URUNGKAN. Gunakan id sebagai key.
5. Simulasikan pemuatan bertahap: tampilkan 15 data pertama, lalu muat 15 berikutnya saat mendekati bawah dengan `ScrollController` (beri jeda `Future.delayed`).
6. Tangani keadaan loading awal, daftar kosong, dan hasil pencarian kosong dengan tampilan yang berbeda.
7. Tambahkan `RefreshIndicator` untuk mereset daftar ke halaman pertama.
8. Buat tab kedua (atau layar terpisah) berisi dosen favorit dalam `GridView.builder` dengan `SliverGridDelegateWithMaxCrossAxisExtent`.
9. Pastikan controller di-dispose dan `flutter analyze` bersih.

Output yang Diharapkan

Direktori dosen yang lancar saat digulir, bisa dicari dan difilter secara instan, mendukung swipe favorit/hapus dengan urungkan, memuat data bertahap, dan menampilkan pesan yang jelas saat loading atau kosong.

Rubrik Penilaian

ListView.builder + ListTile yang benar (20%), Pencarian & filter tanpa merusak data asli (20%), Dismissible dengan key unik, konfirmasi, dan urungkan (20%), Infinite scroll dengan penjaga

loading ganda (15%), Loading/empty state + RefreshIndicator (15%), GridView favorit dan kerapian kode (10%).

Kuis Pemahaman

1. Mengapa ListView.builder jauh lebih hemat memori dibanding ListView biasa saat menampilkan 10.000 data?

A. Karena ListView.builder hanya merender item yang saat ini tampak di layar viewport dan mendaur ulang item yang terlewat

B. Karena ListView.builder mengompres resolusi gambar menjadi hitam putih

C. Karena ListView.builder menghapus data dari RAM secara permanen

D. Karena ListView.builder dijalankan di server bukan di HP pengguna

2. Properti SliverAppBar manakah yang jika diatur bernilai 'true' akan membuat judul AppBar tetap menempel di atas layar saat pengguna menggulir konten jauh ke bawah?

A. floating: true

B. pinned: true

C. snap: true

D. stretch: true

3. Kapan SingleChildScrollView lebih tepat dipakai daripada ListView.builder?

A. Untuk halaman dengan isi terbatas seperti form atau profil yang hanya perlu bisa digulir

B. Untuk menampilkan 10.000 data dari API

C. Untuk membuat grid dua kolom

D. Untuk membuat item yang bisa digeser untuk dihapus

4. Muncul error "Vertical viewport was given unbounded height" saat ListView diletakkan di dalam Column. Perbaikan yang tepat adalah...

A. Membungkus ListView dengan Expanded

B. Mengganti Column dengan Row

C. Menghapus itemCount

D. Membungkus Column dengan SingleChildScrollView

5. Mengapa key pada Dismissible sebaiknya memakai id unik data (misalnya ValueKey(item.id)) dan bukan index?

A. Karena index bergeser setelah item dihapus sehingga Flutter bisa salah mengenali item

B. Karena index tidak boleh berupa angka

C. Karena ValueKey membuat animasi lebih cepat

D. Karena Dismissible tidak menerima tipe int

6. Apa syarat fungsi onRefresh pada RefreshIndicator?

A. Harus mengembalikan Future; indikator berputar sampai Future selesai

B. Harus mengembalikan bool

C. Harus memanggil Navigator.pop

D. Harus bersifat synchronous tanpa async

7. Pada infinite scroll, mengapa diperlukan variabel penjaga seperti _sedangMemuat?

A. Agar halaman yang sama tidak dimuat berkali-kali karena listener scroll terpanggil sangat sering

B. Agar ListView tidak bisa digulir

C. Agar ScrollController tidak perlu di-dispose

D. Agar data otomatis tersimpan ke database

8. Widget Text ingin diletakkan di antara SliverAppBar dan SliverList di dalam CustomScrollView. Caranya adalah...

A. Membungkus Text dengan SliverToBoxAdapter

B. Meletakkan Text langsung di daftar slivers

C. Membungkus Text dengan Expanded

D. Memindahkan Text ke parameter appBar Scaffold

Kunci Jawaban & Pembahasan

1. A — ListView.builder menggunakan konsep viewport lazy recycling. Widget anak hanya dibuat saat mendekati layar yang terlihat dan dibuang/digunakan ulang saat keluar layar, menjaga konsumsi memori tetap stabil.

2. B — pinned: true memastikan AppBar tidak akan hilang saat pengguna menggulir ke bawah, melainkan tetap terlihat (menempel) di bagian atas layar.

3. A — SingleChildScrollView membuat seluruh isinya sekaligus, sehingga cocok untuk konten terbatas. Untuk data banyak gunakan ListView.builder yang lazy.

4. A — Column memberi tinggi tak terbatas kepada anaknya, sedangkan ListView ingin mengisi seluruh tinggi yang tersedia. Expanded memberi ListView tinggi sisa yang jelas.

5. A — Setelah item dihapus, index item-item lain berubah. Key berbasis index membuat Flutter mengira item lain adalah item yang sama, memicu error atau tampilan yang salah.

6. A — onRefresh bertipe Future<void> Function(). RefreshIndicator menampilkan lingkaran loading sampai Future tersebut selesai.

7. A — Listener ScrollController dipanggil setiap perubahan posisi gulir. Tanpa penjaga, beberapa permintaan halaman yang sama terkirim bersamaan sehingga data menjadi dobel.

8. A — Semua anak CustomScrollView harus berupa sliver. Widget biasa dibungkus SliverToBoxAdapter agar bisa ditempatkan di antara sliver lain.